

Clustering Analysis of Global Prosperity Indicators

Sandaru Welikala, Laura Granada, Diego Diaz, Julio Real

2024-11-09

This project seeks to investigate clustering methods to uncover meaningful patterns and groupings within two distinct datasets, each chosen to emphasize various clustering techniques. Specifically, in this portion of the project, hierarchical clustering will be applied using different types of linkage methods, with a focus on analyzing the Global Prosperity and Development Indicators dataset.

The Global Prosperity and Development Indicators dataset includes a wide range of metrics designed to measure the prosperity and development of countries across the globe. It covers various domains such as governance, economic quality, safety and security, personal freedom, social capital, education, health, and living conditions. The data were collected from reputable global sources, ensuring high accuracy and reliability. The dataset features country-level aggregated scores and rankings, providing a comprehensive overview of performance in each domain. These indicators are commonly used for comparative analysis, policy-making, and research on global development trends. The data reflect the status of each country across multiple dimensions, making it a valuable resource for assessing regional disparities and identifying areas for improvement in global prosperity.

```
library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.4.2
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(clustertend)
```

```
## Package 'clustertend' is deprecated. Use package 'hopkins' instead.
```

```
library(hopkins)
```

```
## Warning: package 'hopkins' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'hopkins'
```

```
## The following object is masked from 'package:clustertend':
```

```
##
```

```
##      hopkins
```

```
library(NbClust)
library(dendextend)
```

```
## Warning: package 'dendextend' was built under R version 4.4.2
```

```
##
## -----
## Welcome to dendextend version 1.19.0
## Type citation('dendextend') for how to cite the package.
##
## Type browseVignettes(package = 'dendextend') for the package vignette.
## The github page is: https://github.com/talgalili/dendextend/
##
## Suggestions and bug-reports can be submitted at: https://github.com/talgalili/dendextend/issues
## You may ask questions at stackoverflow, use the r and dendextend tags:
##   https://stackoverflow.com/questions/tagged/dendextend
##
## To suppress this message use: suppressPackageStartupMessages(library(dendextend))
## -----

##
## Attaching package: 'dendextend'

## The following object is masked from 'package:stats':
##
##   cutree
```

```
library(fpc)
```

```
## Warning: package 'fpc' was built under R version 4.4.2
```

```
library(clValid)
```

```
## Warning: package 'clValid' was built under R version 4.4.2
```

```
## Loading required package: cluster
```

```
library(cluster)
library(ggplot2)
library(GGally)
```

```
## Warning: package 'GGally' was built under R version 4.4.2
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg      ggplot2
```

```
library(tibble)
library(igraph)
```

```
## Warning: package 'igraph' was built under R version 4.4.2
```

```
##
## Attaching package: 'igraph'
```

```
## The following object is masked from 'package:tibble':
##
##   as_data_frame
```

```
## The following object is masked from 'package:clValid':
##
##   clusters
```

```
## The following objects are masked from 'package:stats':
##
##   decompose, spectrum
```

```
## The following object is masked from 'package:base':
##
##   union
```

```
library(reshape2)
```

```
## Warning: package 'reshape2' was built under R version 4.4.2
```

```
library(rnaturalearth)
```

```
## Warning: package 'rnaturalearth' was built under R version 4.4.2
```

```
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.4.2
```

```
## Linking to GEOS 3.12.2, GDAL 3.9.3, PROJ 9.4.1; sf_use_s2() is TRUE
```

```
library(stringr)
```

```
## Warning: package 'stringr' was built under R version 4.4.2
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.2
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:igraph':
##
##   as_data_frame, groups, union

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(visdat)
```

```
## Warning: package 'visdat' was built under R version 4.4.2
```

```
library(gridExtra)
```

```
## Warning: package 'gridExtra' was built under R version 4.4.2
```

```
##
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:dplyr':
##
##   combine
```

```
country <- read.csv("../data/global prosperity and development indicators.csv")
```

Exploratory Data Analysis of the Data Sets

```
head(country)
```

EDA Prosperity

```
##      Country AveragScore SafetySecurity PersonelFreedom Governance
## 1      Denmark      84.55          92.59          94.09      89.45
## 2       Sweden      83.67          90.97          91.90      86.41
## 3       Norway      83.59          93.30          94.10      89.66
## 4       Finland      83.47          89.56          91.96      90.41
## 5 Switzerland      83.42          95.66          87.50      87.67
## 6 Netherlands      82.32          91.19          90.08      87.34
## SocialCapital InvestmentEnvironment EnterpriseConditions
## 1          82.56          82.42          79.64
## 2          78.29          82.81          75.54
## 3          79.03          82.24          75.95
## 4          77.27          84.12          77.25
```

```
## 5      69.14      80.81      83.84
## 6      74.03      84.11      79.09
##      MarketAccessInfrastructure EconomicQuality LivingConditions Health Education
## 1              78.79              76.81              95.77 81.07      87.48
## 2              79.67              76.18              95.33 82.28      85.92
## 3              75.87              77.25              94.70 82.98      85.68
## 4              78.77              70.28              94.46 81.19      88.38
## 5              78.65              79.71              94.66 82.11      87.72
## 6              80.82              74.34              95.86 82.05      86.43
##      NaturalEnvironment
## 1              73.94
## 2              78.74
## 3              72.37
## 4              77.99
## 5              73.60
## 6              62.49
```

```
head(country,2)
```

EDA Prosperity Index

```
##      Country AveragScore SafetySecurity PersonelFreedom Governance SocialCapital
## 1  Denmark      84.55      92.59      94.09      89.45      82.56
## 2   Sweden      83.67      90.97      91.90      86.41      78.29
##      InvestmentEnvironment EnterpriseConditions MarketAccessInfrastructure
## 1              82.42              79.64              78.79
## 2              82.81              75.54              79.67
##      EconomicQuality LivingConditions Health Education NaturalEnvironment
## 1              76.81              95.77 81.07      87.48      73.94
## 2              76.18              95.33 82.28      85.92      78.74
```

```
# Remove non numeric columns
country_new <- country[,-1]
head(country_new, 2)
```

```
##      AveragScore SafetySecurity PersonelFreedom Governance SocialCapital
## 1      84.55      92.59      94.09      89.45      82.56
## 2      83.67      90.97      91.90      86.41      78.29
##      InvestmentEnvironment EnterpriseConditions MarketAccessInfrastructure
## 1              82.42              79.64              78.79
## 2              82.81              75.54              79.67
##      EconomicQuality LivingConditions Health Education NaturalEnvironment
## 1              76.81              95.77 81.07      87.48      73.94
## 2              76.18              95.33 82.28      85.92      78.74
```

```
# Dimensions
paste("The Prosperity Index data numerical subset is comprised of:", dim(country_new)[1], " observations,
```

```
## [1] "The Prosperity Index data numerical subset is comprised of: 167 observations/rows and 13 variables"
```

```
# Summary
summary(country_new)
```

```
##      AveragScore      SafetySecurity      PersonelFreedom      Governance
## Min.      :30.40    Min.      :16.54    Min.      :16.16    Min.      :13.09
## 1st Qu.:47.77    1st Qu.:59.29    1st Qu.:39.65    1st Qu.:37.47
## Median :57.53    Median :68.93    Median :57.17    Median :47.51
## Mean      :58.06    Mean      :67.24    Mean      :56.83    Mean      :50.36
## 3rd Qu.:66.86    3rd Qu.:80.56    3rd Qu.:72.86    3rd Qu.:60.97
## Max.      :84.55    Max.      :96.32    Max.      :94.10    Max.      :90.41
## SocialCapital      InvestmentEnvironment      EnterpriseConditions
## Min.      :23.01    Min.      :21.69          Min.      :20.50
## 1st Qu.:47.82    1st Qu.:40.77          1st Qu.:45.83
## Median :54.39    Median :51.25          Median :53.52
## Mean      :54.47    Mean      :53.01          Mean      :54.79
## 3rd Qu.:60.61    3rd Qu.:64.64          3rd Qu.:62.76
## Max.      :82.56    Max.      :84.99          Max.      :83.84
## MarketAccessInfrastructure      EconomicQuality      LivingConditions      Health
## Min.      :24.23          Min.      :24.46    Min.      :19.21    Min.      :31.95
## 1st Qu.:40.05          1st Qu.:41.30    1st Qu.:55.83    1st Qu.:60.59
## Median :56.59          Median :50.06    Median :74.77    Median :71.38
## Mean      :55.02          Mean      :51.57    Mean      :69.80    Mean      :68.62
## 3rd Qu.:69.06          3rd Qu.:62.55    3rd Qu.:86.97    3rd Qu.:77.34
## Max.      :85.75          Max.      :80.10    Max.      :95.86    Max.      :86.89
##      Education      NaturalEnvironment
## Min.      :16.78    Min.      :33.67
## 1st Qu.:44.35    1st Qu.:50.27
## Median :61.93    Median :55.54
## Mean      :58.72    Mean      :56.23
## 3rd Qu.:74.13    3rd Qu.:61.94
## Max.      :91.44    Max.      :78.74
```

```
# Calculate standard deviation for each numeric column
sds <- apply(country[, sapply(country, is.numeric)], 2, sd)

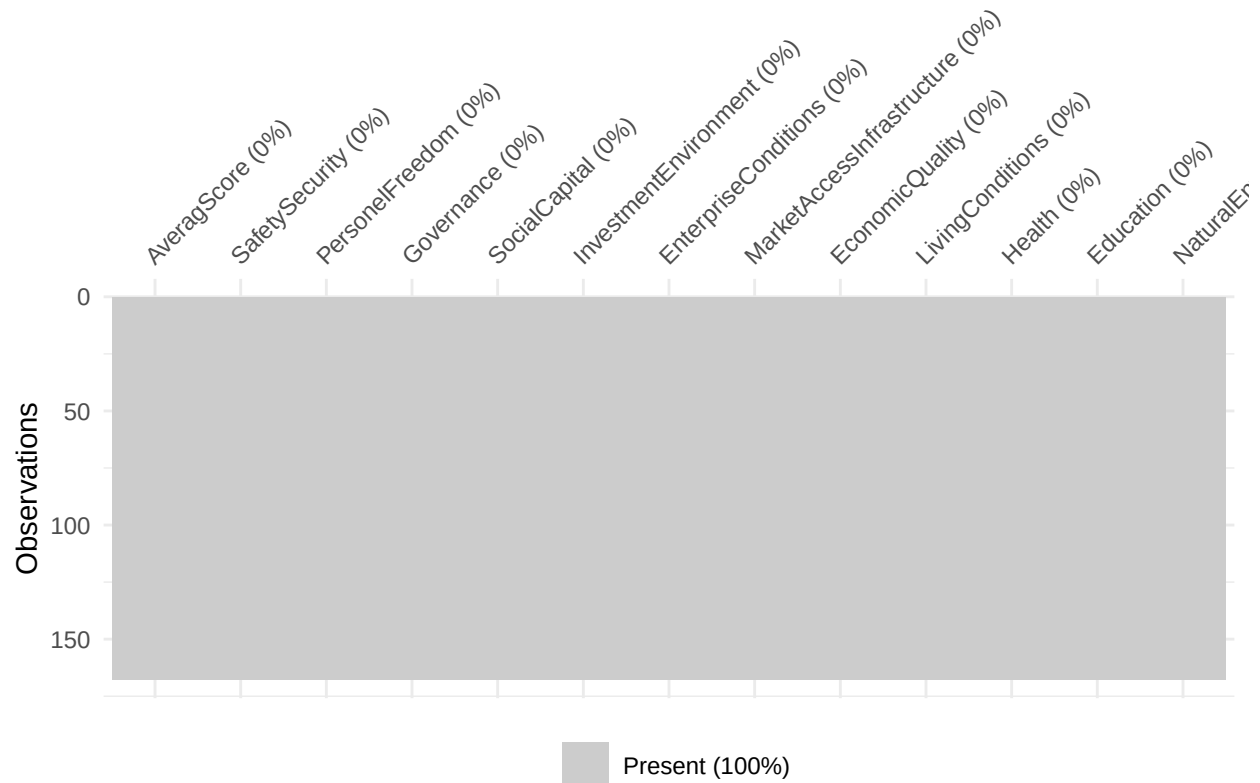
print(sds)
```

```
##      AveragScore      SafetySecurity
##      13.309964      17.542279
##      PersonelFreedom      Governance
##      19.914638      17.747092
##      SocialCapital      InvestmentEnvironment
##      10.350639      16.743723
##      EnterpriseConditions      MarketAccessInfrastructure
##      12.876166      15.849004
##      EconomicQuality      LivingConditions
##      13.427811      19.752677
##      Health      Education
##      11.166266      19.679023
##      NaturalEnvironment
##      9.061262
```

```
# Missing values
anyNA(country_new)
```

```
## [1] FALSE
```

```
vis_miss(country_new)
```



There are no missing values in the data set.

```
# Visualization of data
cor(country_new)
```

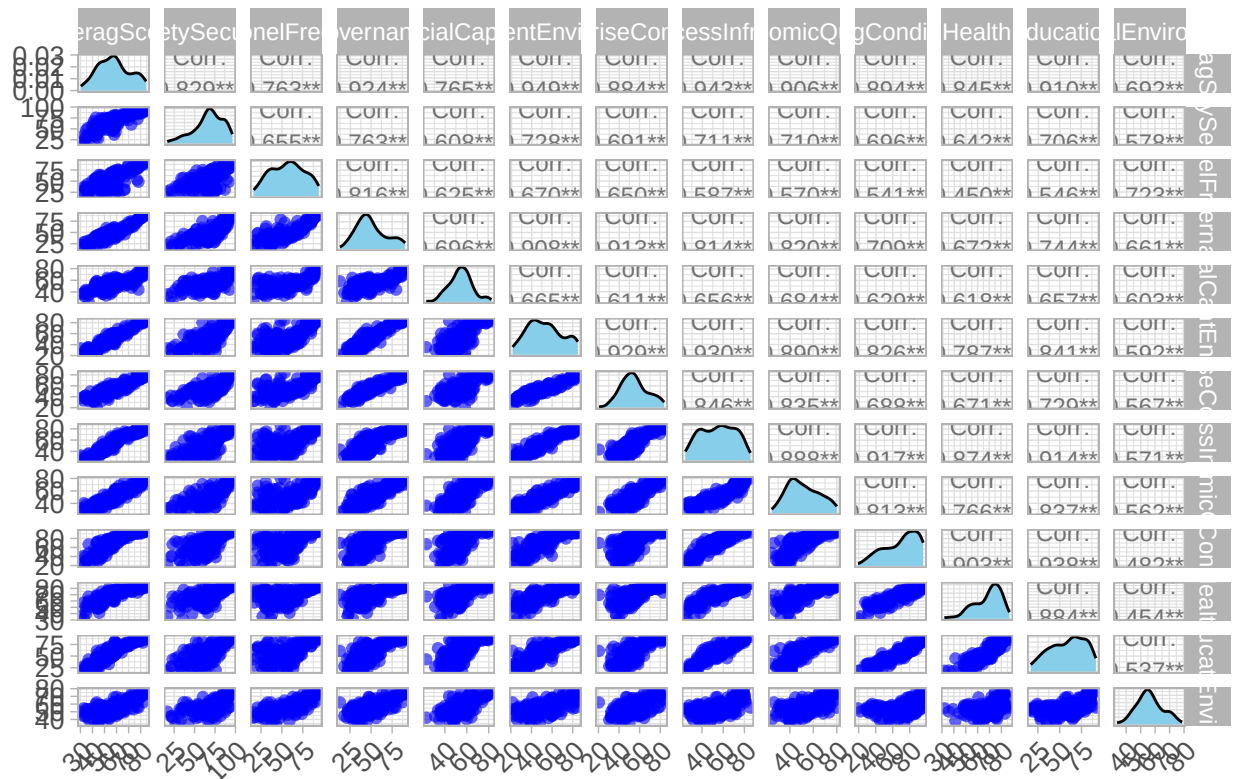
```
##               AveragScore SafetySecurity PersonelFreedom
## AveragScore      1.0000000      0.8286043      0.7630933
## SafetySecurity    0.8286043      1.0000000      0.6550186
## PersonelFreedom   0.7630933      0.6550186      1.0000000
## Governance        0.9240586      0.7625535      0.8163123
## SocialCapital      0.7645955      0.6077455      0.6251797
## InvestmentEnvironment 0.9486494      0.7279673      0.6700443
## EnterpriseConditions 0.8840385      0.6910160      0.6496571
## MarketAccessInfrastructure 0.9433409      0.7112079      0.5869381
## EconomicQuality    0.9062940      0.7098024      0.5697417
## LivingConditions   0.8944886      0.6960774      0.5406233
## Health            0.8453267      0.6419238      0.4495919
```

## Education	0.9102754	0.7060964	0.5456998
## NaturalEnvironment	0.6918733	0.5779618	0.7225107
##	Governance	SocialCapital	InvestmentEnvironment
## AveragScore	0.9240586	0.7645955	0.9486494
## SafetySecurity	0.7625535	0.6077455	0.7279673
## PersonelFreedom	0.8163123	0.6251797	0.6700443
## Governance	1.0000000	0.6964100	0.9078410
## SocialCapital	0.6964100	1.0000000	0.6648110
## InvestmentEnvironment	0.9078410	0.6648110	1.0000000
## EnterpriseConditions	0.9128200	0.6105658	0.9294641
## MarketAccessInfrastructure	0.8143549	0.6561130	0.9296575
## EconomicQuality	0.8195496	0.6841178	0.8903182
## LivingConditions	0.7092157	0.6293916	0.8257216
## Health	0.6718444	0.6182214	0.7873502
## Education	0.7441135	0.6567346	0.8411109
## NaturalEnvironment	0.6606324	0.6030995	0.5915626
##	EnterpriseConditions	MarketAccessInfrastructure	
## AveragScore		0.8840385	0.9433409
## SafetySecurity		0.6910160	0.7112079
## PersonelFreedom		0.6496571	0.5869381
## Governance		0.9128200	0.8143549
## SocialCapital		0.6105658	0.6561130
## InvestmentEnvironment		0.9294641	0.9296575
## EnterpriseConditions		1.0000000	0.8462679
## MarketAccessInfrastructure		0.8462679	1.0000000
## EconomicQuality		0.8349193	0.8879706
## LivingConditions		0.6877950	0.9166084
## Health		0.6714879	0.8737858
## Education		0.7287170	0.9144986
## NaturalEnvironment		0.5669594	0.5706525
##	EconomicQuality	LivingConditions	Health Education
## AveragScore	0.9062940	0.8944886	0.8453267 0.9102754
## SafetySecurity	0.7098024	0.6960774	0.6419238 0.7060964
## PersonelFreedom	0.5697417	0.5406233	0.4495919 0.5456998
## Governance	0.8195496	0.7092157	0.6718444 0.7441135
## SocialCapital	0.6841178	0.6293916	0.6182214 0.6567346
## InvestmentEnvironment	0.8903182	0.8257216	0.7873502 0.8411109
## EnterpriseConditions	0.8349193	0.6877950	0.6714879 0.7287170
## MarketAccessInfrastructure	0.8879706	0.9166084	0.8737858 0.9144986
## EconomicQuality	1.0000000	0.8132584	0.7658000 0.8366795
## LivingConditions	0.8132584	1.0000000	0.9032194 0.9378222
## Health	0.7658000	0.9032194	1.0000000 0.8839161
## Education	0.8366795	0.9378222	0.8839161 1.0000000
## NaturalEnvironment	0.5620172	0.4819822	0.4541838 0.5369733
##	NaturalEnvironment		
## AveragScore	0.6918733		
## SafetySecurity	0.5779618		
## PersonelFreedom	0.7225107		
## Governance	0.6606324		
## SocialCapital	0.6030995		
## InvestmentEnvironment	0.5915626		
## EnterpriseConditions	0.5669594		
## MarketAccessInfrastructure	0.5706525		
## EconomicQuality	0.5620172		


```
## LivingConditions          0.4819822
## Health                   0.4541838
## Education                 0.5369733
## NaturalEnvironment        1.0000000
```

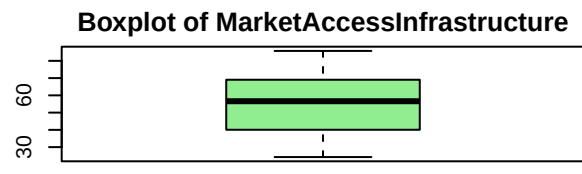
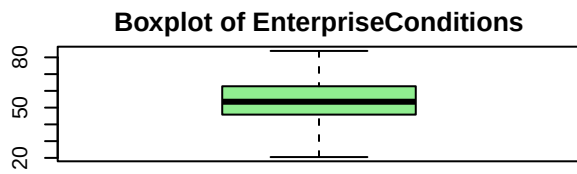
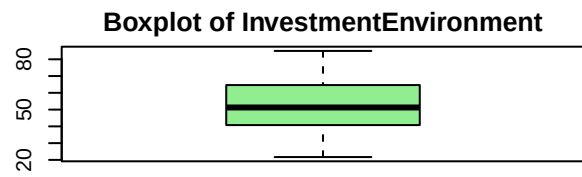
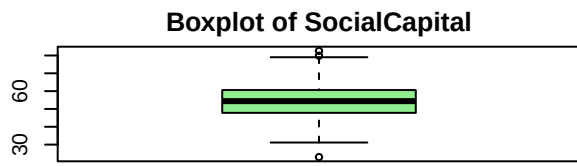
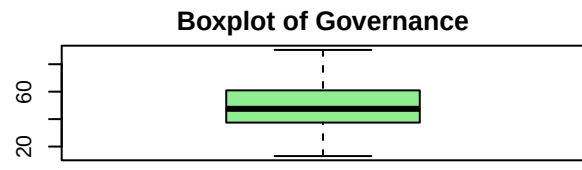
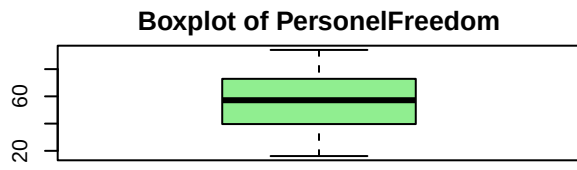
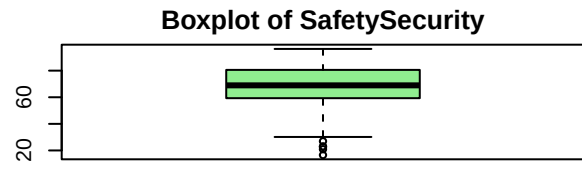
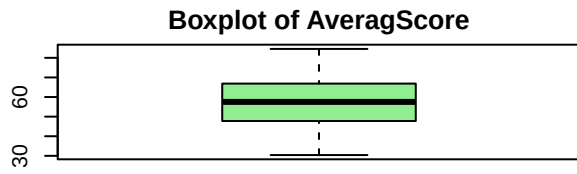
```
# Enhanced ggpairs plot
pairwise_plot <- ggpairs(
  country_new,
  cardinality_threshold = 200,
  upper = list(
    continuous = wrap("cor", size = 3) # Adjust correlation text size
  ),
  lower = list(
    continuous = wrap("points", alpha = 0.6, color = "blue") # Add color and transparency
  ),
  diag = list(
    continuous = wrap("densityDiag", fill = "skyblue") # Colored density plots
  ),
  title = "Pairwise Plot of Prosperity data"
) +
theme_light() + # Apply a light theme for better contrast
theme(
  axis.text.x = element_text(
    angle = 45, hjust = 1, size = 10 # Rotate and adjust x-axis text
  ),
  axis.text.y = element_text(size = 10), # Adjust y-axis text
  plot.title = element_text(
    size = 16, face = "bold" # Title styling
  )
)
pairwise_plot
```

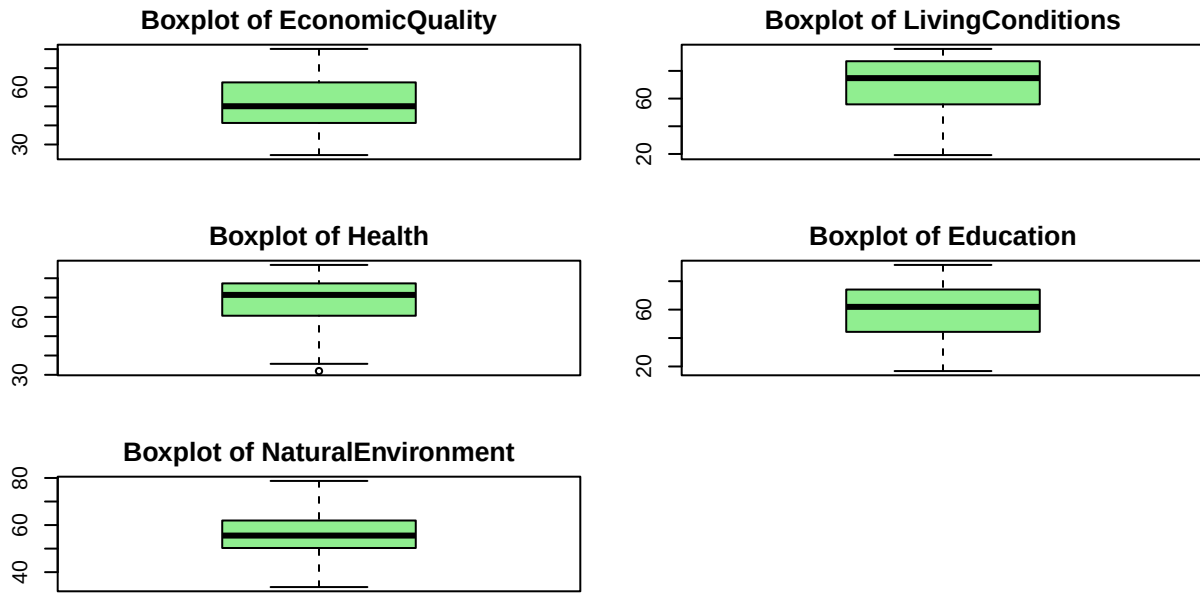
Pairwise Plot of Prosperity data



```
ggsave("ggpairs_plots.png", plot = pairwise_plot, width = 14, height = 10)
```

```
# Box plots to detect outliers
par(mfrow = c(4, 2), mar = c(2, 2, 2, 2))
for (i in 2:(ncol(country))) {
  boxplot(country[[i]], main = paste("Boxplot of", colnames(country)[i]), col = "lightgreen")
}
```

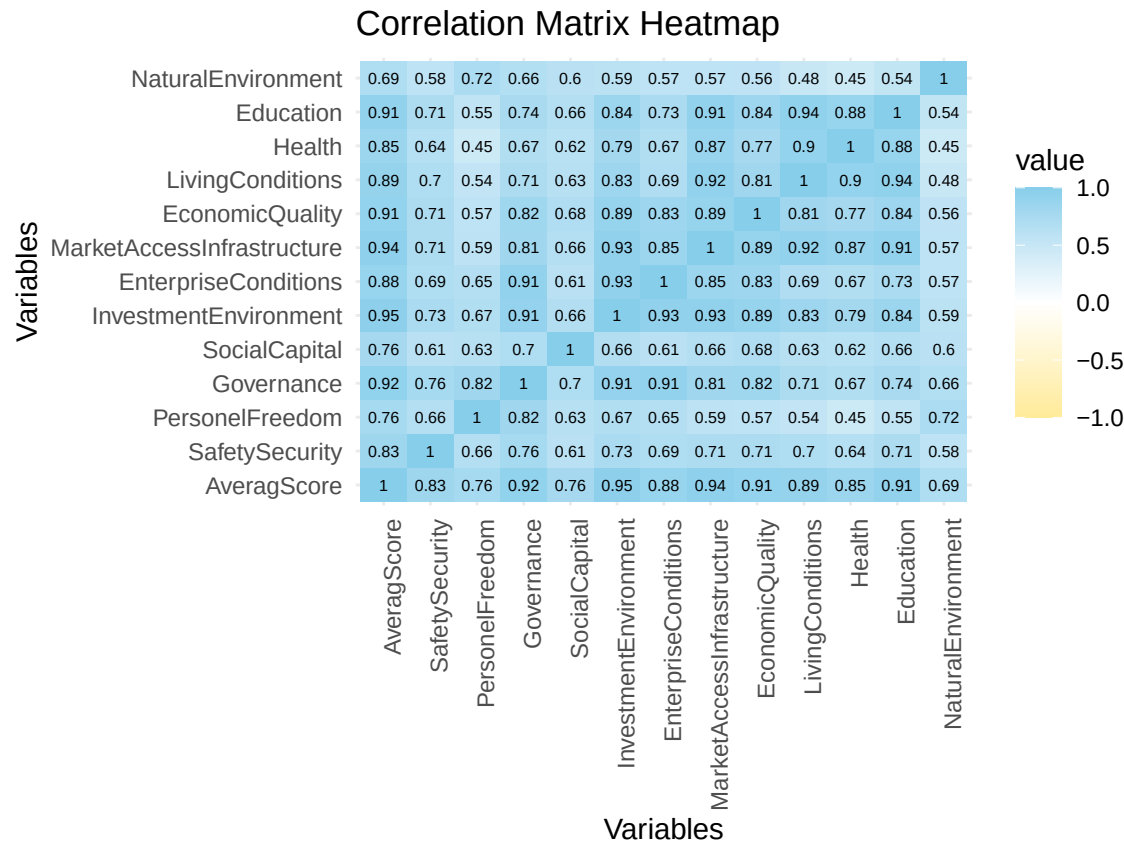




```
# Calculate the correlation matrix
corr_matrix <- cor(country_new)

# Reshape the correlation matrix for ggplot
corr_melted <- melt(corr_matrix)

ggplot(data = corr_melted, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "#FFEB99", mid = "white", high = "skyblue", midpoint = 0, limit = c(-1, 1)) +
  geom_text(aes(label = round(value, 2)), color = "black", size = 2) + # Add correlation values inside
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1),
        axis.text.y = element_text(angle = 0)) +
  labs(title = "Correlation Matrix Heatmap", x = "Variables", y = "Variables")
```



```
# Variables where correlation is greater than 0.90
high_corr_pairs <- corr_melted[corr_melted$value > 0.90 & corr_melted$Var1 != corr_melted$Var2, ]

# Create a unique representation of the pair to handle duplicates, (Var1, Var2) and (Var2, Var1) are covered
high_corr_pairs$pair <- apply(high_corr_pairs[, c("Var1", "Var2")], 1, function(x) paste(sort(x), collapse="_"))

# Remove duplicates by selecting unique pairs
high_corr_pairs_unique <- high_corr_pairs[!duplicated(high_corr_pairs$pair), ]

# Now display the cleaned data without duplicates
high_corr_pairs_unique <- high_corr_pairs_unique[, c("Var1", "Var2", "value")]

# Display the results
print(high_corr_pairs_unique)
```

```
##           Var1           Var2      value
## 4      Governance AveragScore 0.9240586
## 6 InvestmentEnvironment AveragScore 0.9486494
## 8 MarketAccessInfrastructure AveragScore 0.9433409
## 9      EconomicQuality AveragScore 0.9062940
## 12      Education AveragScore 0.9102754
## 45 InvestmentEnvironment Governance 0.9078410
## 46 EnterpriseConditions Governance 0.9128200
## 72 EnterpriseConditions InvestmentEnvironment 0.9294641
## 73 MarketAccessInfrastructure InvestmentEnvironment 0.9296575
## 101 LivingConditions MarketAccessInfrastructure 0.9166084
```

## 103	Education	MarketAccessInfrastructure	0.9144986
## 128	Health	LivingConditions	0.9032194
## 129	Education	LivingConditions	0.9378222

Variable distribution: - All variables present a semi normal distribution. - *Average*, *Personal Freedom*, *Investment Environment*, present a slight bi modal pattern. - *Safety Security*, *Living Conditions*, *Health* and *Education* are left skewed.

AverageScore: The distribution appears to be fairly symmetrical, with a peak around the mid-to-high range, suggesting that most observations fall around the middle, with fewer extreme low or high scores. This is characteristic of a somewhat normal distribution.

SafetySecurity: This variable shows a slightly left skewed distribution. This indicates that many countries in the dataset have relatively high safety and security levels, while fewer countries score on the lower end.

PersonalFreedom: The distribution appears to be fairly symmetrical. This indicates moderate variability, with a majority of countries having mid-level personal freedom scores but some reaching high freedom levels.

Governance: The variable distribution shows a slightly right skew. This indicates that many countries in the dataset have a score lower on governance quality, with fewer achieving high governance scores.

SocialCapital: The variable distribution appears to be moderately symmetrical with a peak in the middle range. This implies that most countries have average levels of social capital, with fewer countries at the extreme ends.

InvestmentEnvironment: This variable has a slightly right skewed distribution with peaks in the mid-to-high range. This suggests that many countries have moderate to high scores in this areas, with fewer outliers on the lower end.

Enterprise Conditions: The distribution appears to be fairly symmetrical, with a peak around the mid-to-high range, suggesting that most observations fall around the middle, with fewer extreme low or high scores. This is characteristic of a somewhat normal distribution.

MarketAccessInfrastructure: This variable has a slightly right skewed distribution with peaks in the mid-to-high range. This suggests that many countries have moderate to high scores in this areas, with fewer outliers on the lower end.

EconomicQuality: The distribution is somewhat symmetrical, centred around the middle range. This implies that most countries have average economic quality scores, with fewer scoring extremely high or low.

LivingConditions: LivingConditions has a right-skewed distribution, with a concentration of scores in the upper range. This suggests that many countries in the dataset have relatively good living conditions, with a smaller proportion of countries on the lower end.

Health: This distribution is also right-skewed, with a peak in the upper range, indicating that many countries have high health scores. Few countries fall into the lower health score range.

Education: Similar to Health, the Education variable shows a right-skewed distribution with a concentration of high scores, suggesting that education levels are generally high across the dataset.

NaturalEnvironment: The histogram for Natural Environment is more spread out and possibly left-skewed, with a concentration in the mid-to-lower range. This suggests that while many countries have average to lower scores for natural environment, fewer have extremely high scores.

Correlation: There is a *very high/strong* (>.9) correlation between some pairs of variables, such as: - *Investment Environment* - *Governance* (.908) - *Enterprise Conditions* - *Governance* (.913) - *Investment Environment* - *Enterprise Conditions* (.929) - *Investment Environment* - *Market Access Infrastructure* (.930) - *Living* - *Market Access Infrastructure* (.917) - *Market Access Infrastructure* - *Education* (.914) - *Education* - *Living Conditions* (.938)

The *Average Score* is highly correlated to all variables, presumably because it is composed of all the variables' average.

The rest of the variables share a relatively mild correlation between each other in a range 0.4495919 to 0.8839161.

Data Set Description

This dataset contains rankings and indicators from the *2023 Legatum Prosperity Index*, assessing countries based on various dimensions of prosperity and development. It includes measures of safety, freedom, governance, economic quality, and more. The data is valuable for identifying patterns of prosperity and clustering countries based on development indicators.

```
# Create a table with each variable's information
variables <- tibble(
  Feature = c("AveragScore", "SafetySecurity", "PersonalFreedom", "Governance", "SocialCapital",
    "InvestmentEnvironment", "EnterpriseConditions", "MarketAccessInfrastructure",
    "EconomicQuality", "LivingConditions", "Health", "Education", "NaturalEnvironment"),
  Range = c("30.40 - 84.55", "16.54 - 96.32", "16.16 - 94.10", "13.09 - 90.41", "23.01 - 82.56",
    "21.69 - 84.99", "20.50 - 83.84", "24.23 - 85.75", "24.46 - 80.10", "19.21 - 95.86",
    "31.95 - 86.89", "16.78 - 91.44", "33.67 - 78.74"),
  Mean = c(58.06, 67.24, 56.83, 50.36, 54.47, 53.01, 54.79, 55.02, 51.57, 69.80, 68.62, 58.72, 56.23),
  Description = c("The overall average score of the country across all indicators.", "Measures freedom :
    "Quality of democracy, rule of law, and government effectiveness.", "Strength of pers
    "Business environment and market competition.", "Ease of trade and quality of infrast
    "Macroeconomic stability and employment quality. ", "Standard of living and access to
    "Quality and accessibility of education.", "Environmental quality and sustainability.
  Units_of_measurement = c("Points", "Points", "Points", "Points", "Points",
    "Points", "Points", "Points", "Points", "Points",
    "Points", "Points", "Points")
)

# Print the table
print(variables)
```

```
## # A tibble: 13 x 5
##   Feature                                Range      Mean Description Units_of_measurement
##   <chr>                                <chr>      <dbl> <chr>         <chr>
## 1 AveragScore                        30.40 - 84~  58.1 "The overa~ Points
## 2 SafetySecurity                    16.54 - 96~  67.2 "Measures ~ Points
## 3 PersonalFreedom                  16.16 - 94~  56.8 "Assesses ~ Points
## 4 Governance                        13.09 - 90~  50.4 "Quality o~ Points
## 5 SocialCapital                    23.01 - 82~  54.5 "Strength ~ Points
## 6 InvestmentEnvironment             21.69 - 84~  53.0 "Condition~ Points
## 7 EnterpriseConditions              20.50 - 83~  54.8 "Business ~ Points
## 8 MarketAccessInfrastructure       24.23 - 85~  55.0 "Ease of t~ Points
## 9 EconomicQuality                  24.46 - 80~  51.6 "Macroecon~ Points
## 10 LivingConditions                 19.21 - 95~  69.8 "Standard ~ Points
## 11 Health                           31.95 - 86~  68.6 "Measures ~ Points
## 12 Education                        16.78 - 91~  58.7 "Quality a~ Points
## 13 NaturalEnvironment              33.67 - 78~  56.2 "Environme~ Points
```

```
View(variables)
```

Assesing Clustering Tendency

Before applying clustering methods, it's essential to determine if the data contains meaningful clusters. This step prevents misinterpretation, as clustering algorithms will produce clusters regardless of whether they are meaningful or not. The purpose of assessing clustering tendency is to determine if the dataset has non-random structure that justifies clustering.

There are two methods for **Assessing Cluster Tendency**: **Hopkins Statistic** is a common method for assessing clustering tendency by comparing the spatial randomness of the data. To calculate the *Hopkins statistic*, a subset of points is randomly chosen, and for each point, the distance to its nearest neighbour within the data and the distance to a synthetic random point are calculated. The ratio of these distances forms the basis for the *Hopkins statistic*. * Values closer to 1 indicate a higher likelihood of clustering structure. * Values around 0.5 indicate the data likely doesn't have a cluster structure (randomly distributed). * Values close to 0 indicate a highly uniform distribution.

Visual Assessment of Cluster Tendency (VAT) approach creates an ordered dissimilarity image (ODI), which visually highlights clustering structure by displaying similar objects in consecutive order. Square-shaped blocks along the diagonal in the image suggest potential clusters.

Preparation

```
# Standardize the data set
country_scaled <- country.scaled <- scale(country_new)
```

Hopkins Statistic

```
# Compute Hopkins statistic
hopkins_result <- get_clust_tendency(country_scaled, n = nrow(country_scaled) - 1, graph = FALSE)
hopkins_stat <- hopkins_result$hopkins_stat

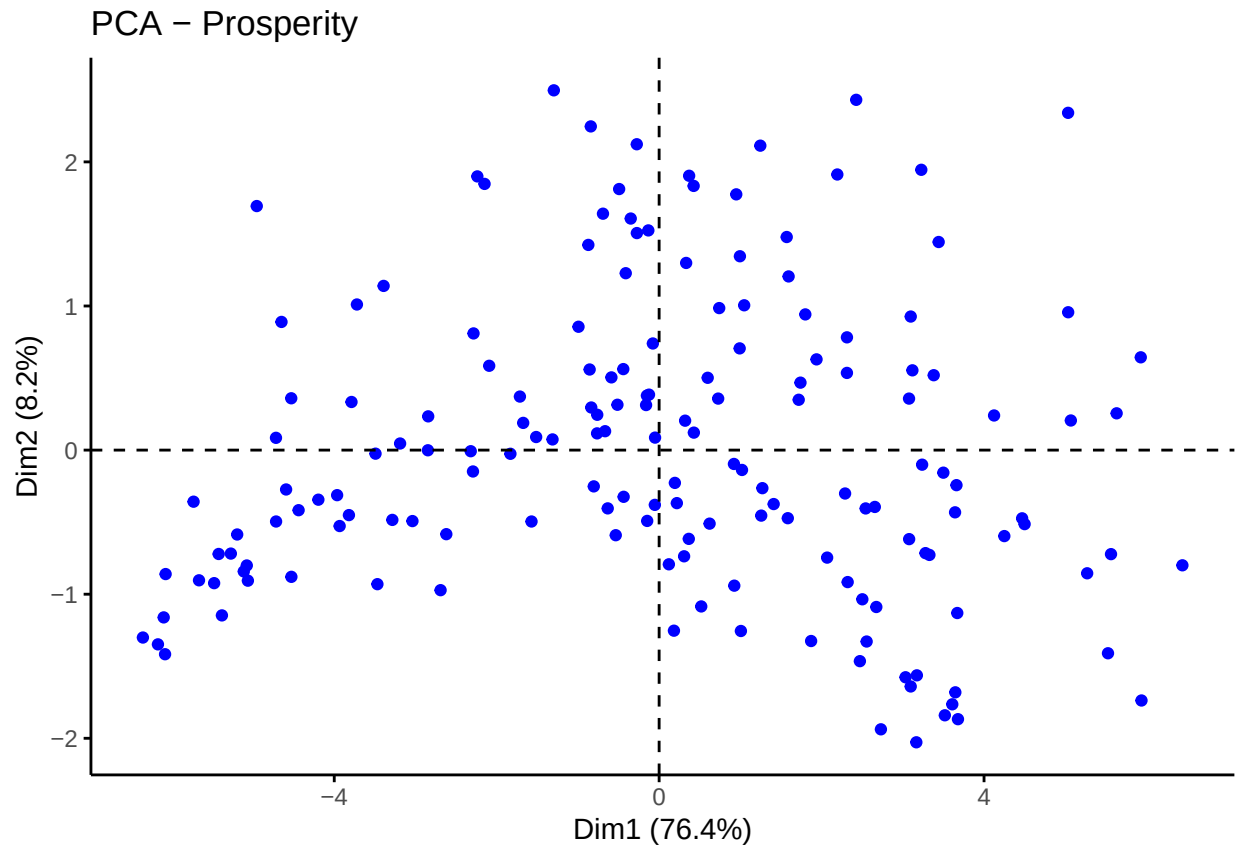
paste("The Hopkins Statistic has a value of:", hopkins_stat)
```

```
## [1] "The Hopkins Statistic has a value of: 0.755996478403451"
```

The Hopkins Statistic has a value of: 0.755996478403451 meaning the data has a moderately high likelihood of clustering structure.

3. Visual Inspection with PCA

```
# Plot PCA for cluster tendency visualization
fviz_pca_ind(prcomp(country_scaled),
  title = "PCA - Prosperity",
  geom = "point",
  col.ind = "blue",
  ggtheme = theme_classic())
```

‘Dim1’ and ‘Dim2’ account for 84.6% of the variance, indicating that these two components capture a significant amount of the data’s structure, allowing to view most of the data’s variability in this 2D plot. This plot suggests that the data has some spread but no strong, obvious clustering pattern in the two dimensions shown.

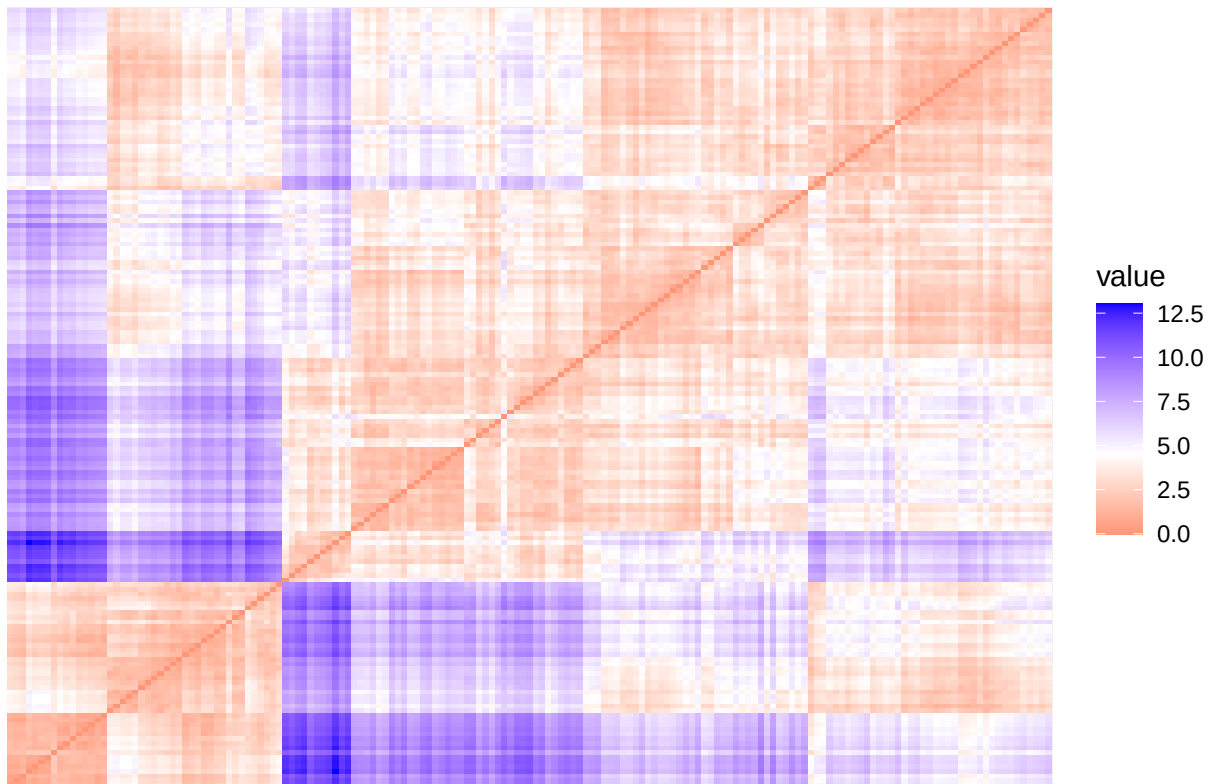
3. Visual Assessment of Clustering Tendency (VAT)

Firstly, we calculate the *distance matrix* on the scaled data set.

```
# Calculate distance matrix
diss_matrix <- dist(country_scaled, method = "euclidean")

# Compute and visualize the dissimilarity matrix
fviz_dist(diss_matrix, show_labels = FALSE)+
  labs(title = "VAT - Distance Matrix Heatmap")
```

VAT – Distance Matrix Heatmap



The dissimilarity matrix image confirms that there is a cluster structure in the “Global Country Development & Prosperity Index” data (apparently 3 clusters). There are regions with high similarity (clusters) among some observations, as evidenced by the block-like patterns along the diagonal.

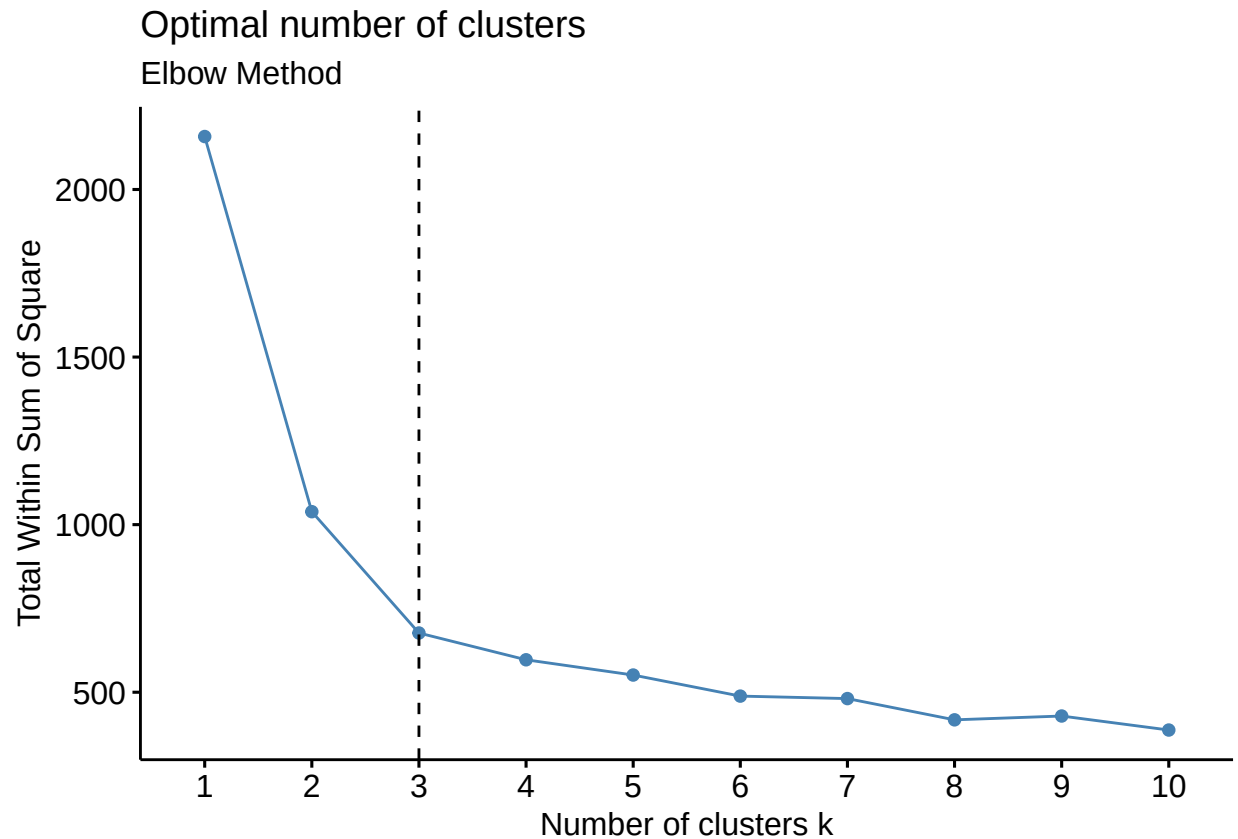
In conclusion, with a *Hopkins* value of approximately 0.756, we can infer that the dataset likely contains clusters, making it suitable for clustering analysis. The dimension reduction capturing variability (PCA) show a slight pattern like structure. The dissimilarity matrix heat map shows there are regions with high similarity, pointing at the existence of clusters.

Determining the number of clusters

Elbow Method

The Elbow Method involves plotting the within-cluster sum of squares (WSS) against the number of clusters (k). The WSS will decrease as the number of clusters increases, but at some point, the decrease becomes marginal. This point is called the “elbow,” and it indicates the optimal number of clusters.

```
# Compute the within-cluster sum of squares for different numbers of clusters
set.seed(123)
plot1<-fviz_nbclust(country_scaled, kmeans, method = "wss") + geom_vline(xintercept = 3, linetype = 2)+
  labs(subtitle = "Elbow Method")
plot1
```

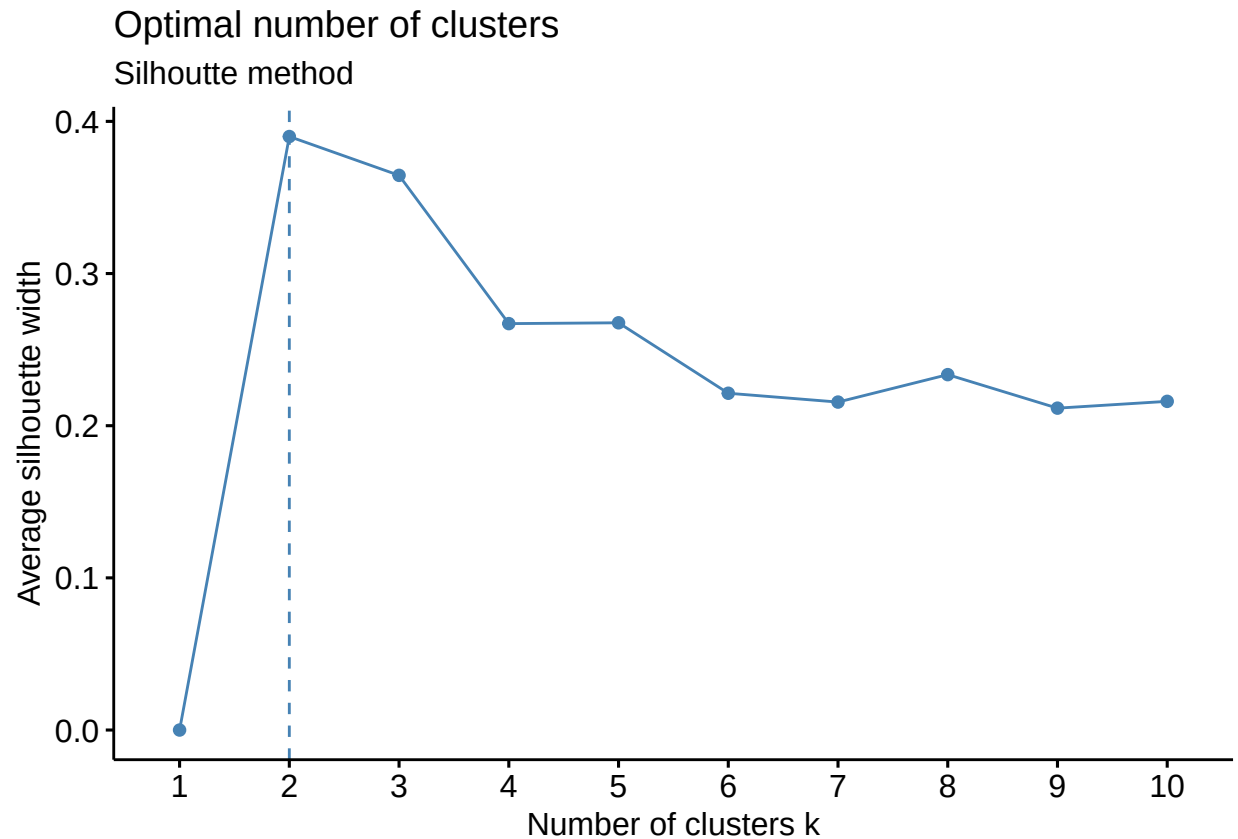


A visual inspection of the possible clusters indicates the elbow is present at $K = 3$.

Silhouette Method

The *Silhouette Method* calculates the average silhouette width for different numbers of clusters. The silhouette width measures how similar each point is to its own cluster compared to other clusters. Higher values indicate better-defined clusters. The optimal number of clusters is where the silhouette width is highest.

```
plot2<-fviz_nbclust(country_scaled, kmeans,method = "silhouette")+  
  labs(subtitle = "Silhoutte method")  
plot2
```

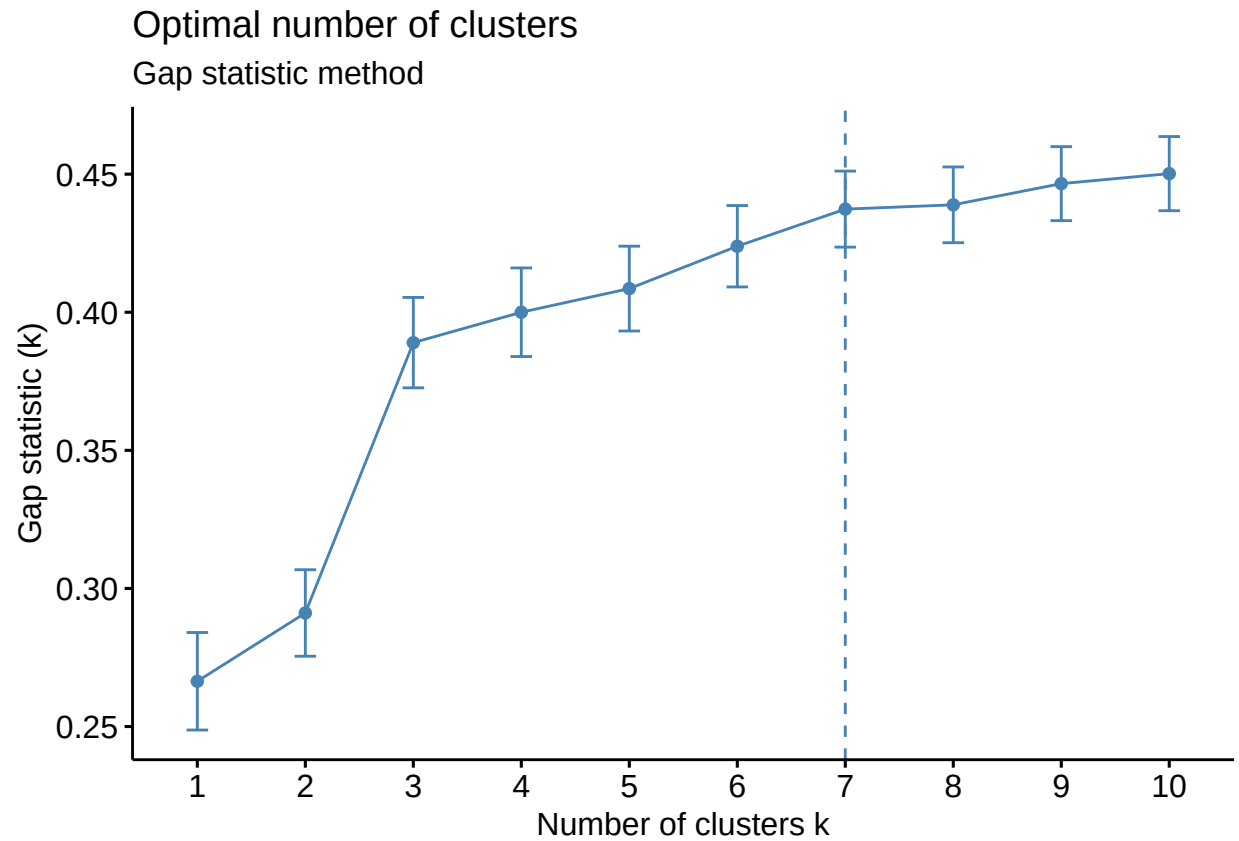


According to the *Silhouette Method* the silhouette width is largest at $K = 2$.

Gap Statistic

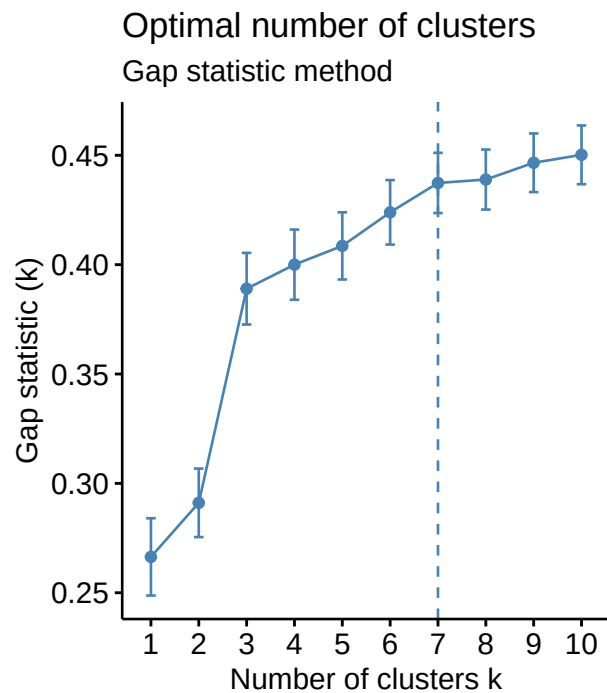
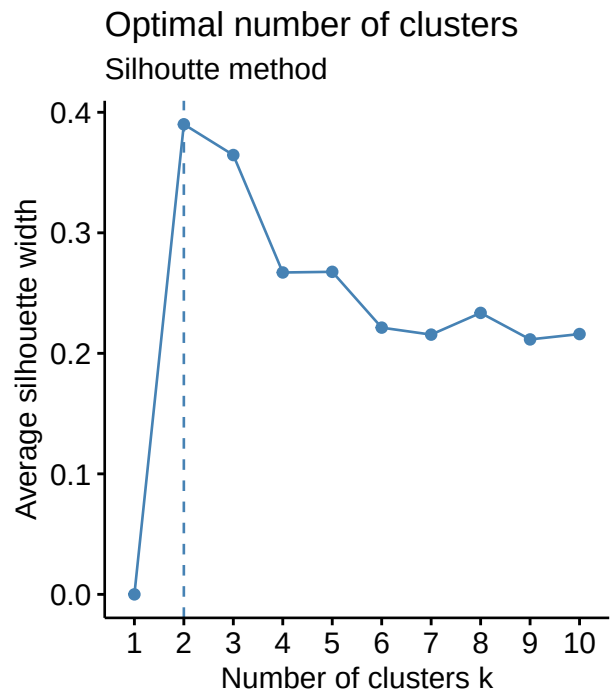
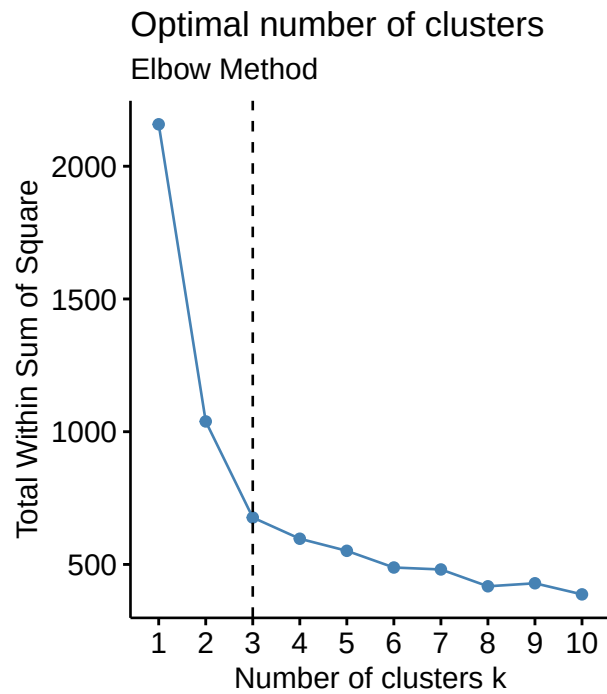
The *Gap Statistic* compares the total within-cluster variation for different numbers of clusters with that expected under a null reference distribution (random, uniform data). The optimal number of clusters is where the gap statistic is maximized.

```
set.seed(123)
plot3<-fviz_nbclust(country_scaled, FUN = kmeans, nstart = 25, method = "gap_stat", nboot = 50)+
  labs(subtitle = "Gap statistic method")
plot3
```



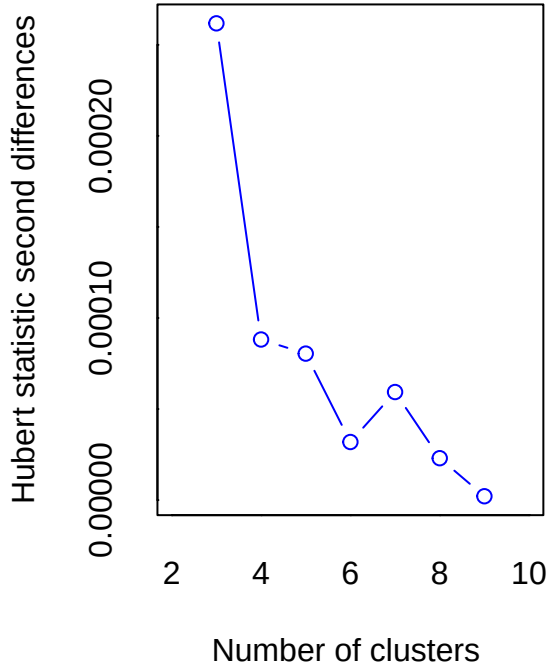
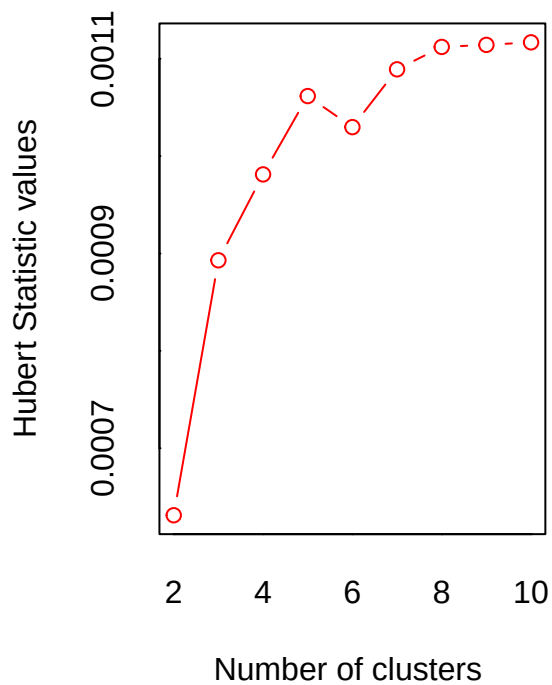
According to the *Gap Statistic* the maximized gap is largest at $K = 7$.

```
# Arrange all plots into a grid
g <- grid.arrange(plot1, plot2, plot3,
  ncol = 2)
```

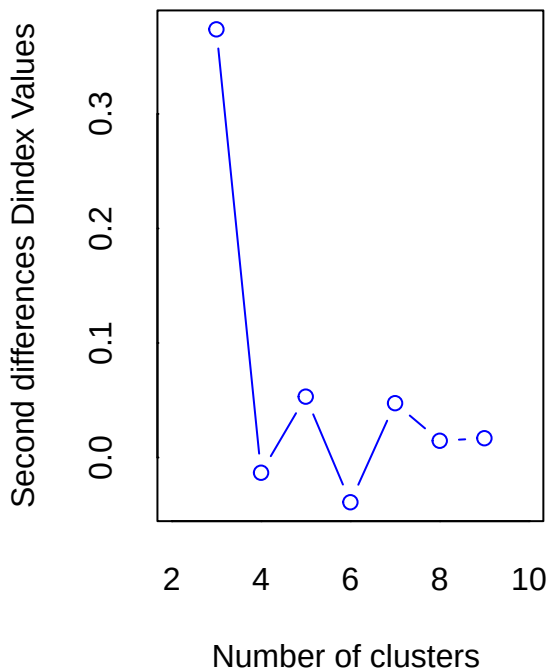
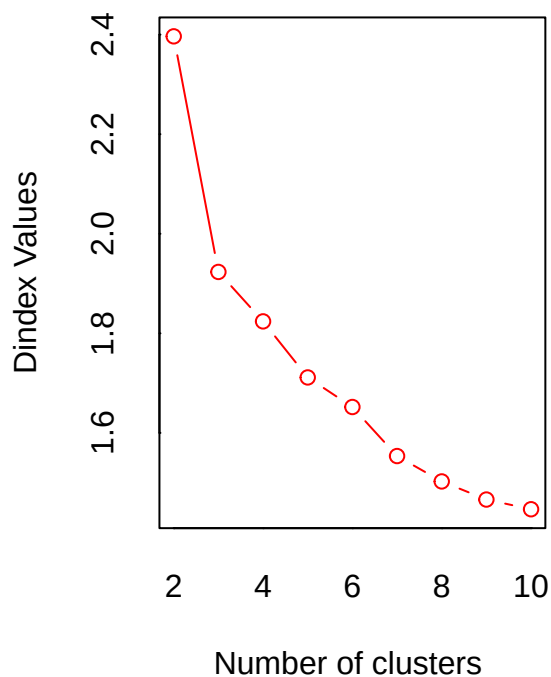


Compute-Nbclust

```
nb <- NbClust(country_scaled, distance = "euclidean", min.nc = 2, max.nc = 10,
              method = "kmeans")
```



```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##       In the plot of Hubert index, we seek a significant knee that corresponds to a
##       significant increase of the value of the measure i.e the significant peak in Hubert
##       index second differences plot.
##
```

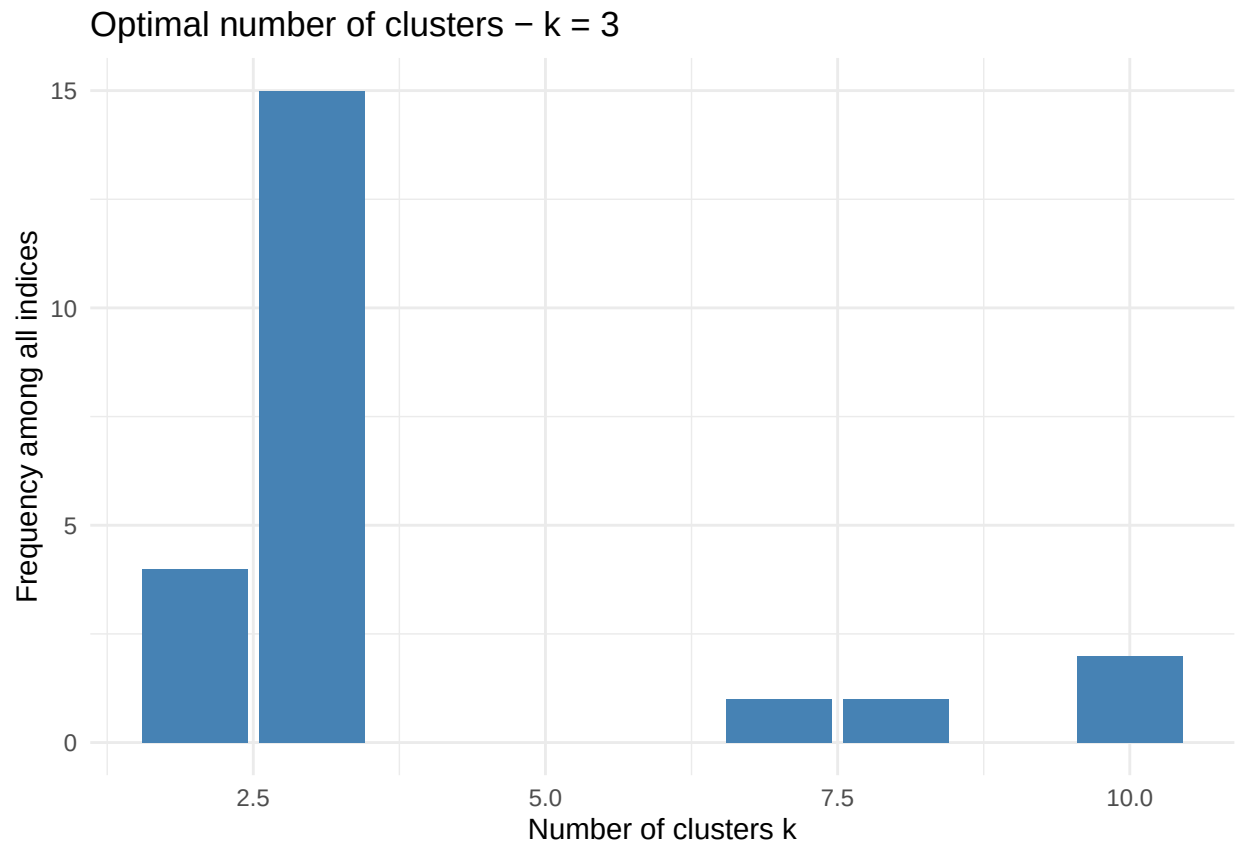


```
## *** : The D index is a graphical method of determining the number of clusters.
##           In the plot of D index, we seek a significant knee (the significant peak in Dindex
##           second differences plot) that corresponds to a significant increase of the value of
##           the measure.
##
## *****
## * Among all indices:
## * 4 proposed 2 as the best number of clusters
## * 15 proposed 3 as the best number of clusters
## * 1 proposed 7 as the best number of clusters
## * 1 proposed 8 as the best number of clusters
## * 2 proposed 10 as the best number of clusters
##
##           ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  3
##
## *****
```

```
data <- data.frame(
  clusters = c(2, 3, 7, 8, 10),
  frequency = c(4, 15, 1, 1, 2)
)
```



```
ggplot(data, aes(x = clusters, y = frequency)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(
    title = "Optimal number of clusters - k = 3",
    x = "Number of clusters k",
    y = "Frequency among all indices"
  ) +
  theme_minimal()
```



After using the two functions (`fviz_nbclust` and `NbClust`) for determining the optimal numbers of clusters we can get following results:

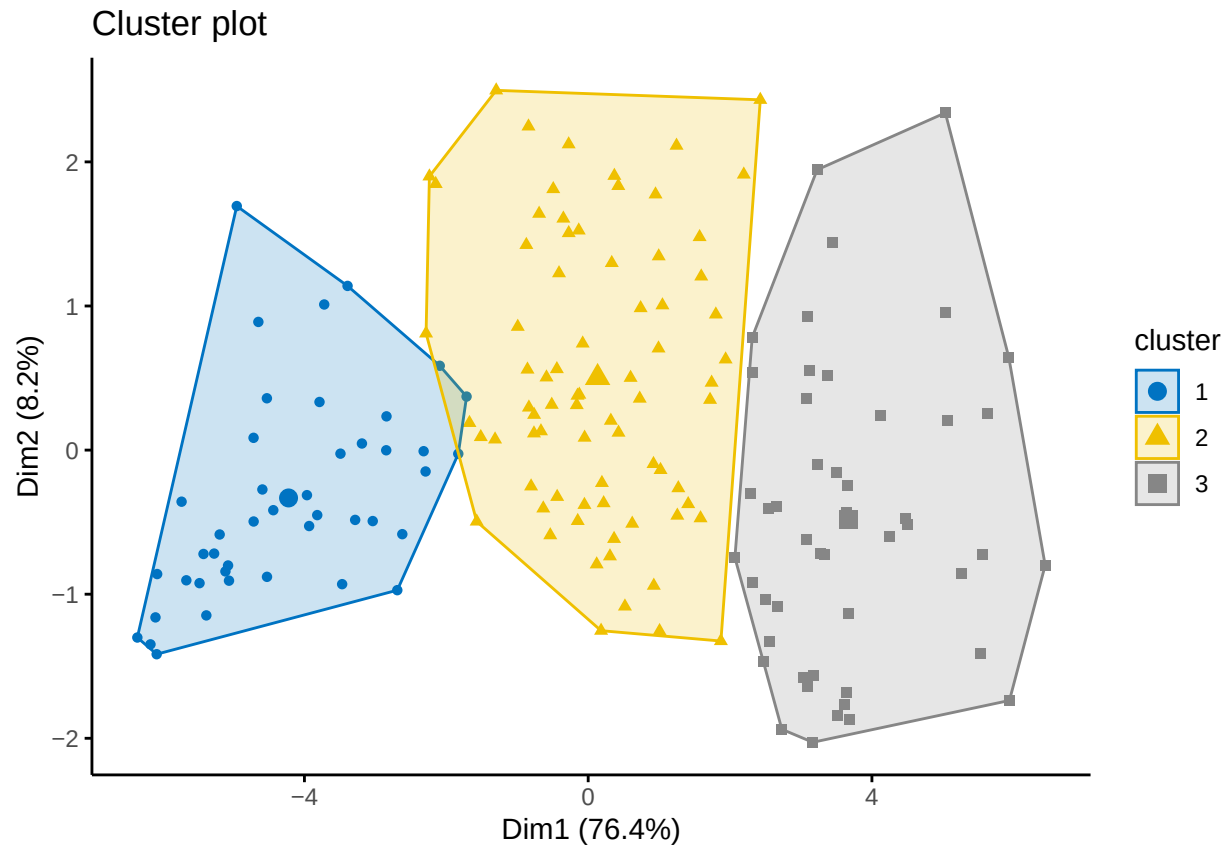
Elbow method: 3 Clusters Silhouette method: 2 Clusters Gap Statistic method: 7 Clusters `NbClust` function: 3 Clusters

Cluster Selection We are going to choose our optimal numbers of clusters as 3, considering that 3 is the numbers of clusters obtained in the Elbow method and the `Nbclust` function.

```
set.seed(123)

hc.res <- eclust(country_scaled, "hclust", k = 3, graph = FALSE)

fviz_cluster(list(data = country_scaled, cluster = hc.res$cluster),
  ellipse.type = "convex", geom = "point", stand = FALSE,
  palette = "jco", ggtheme = theme_classic())
```



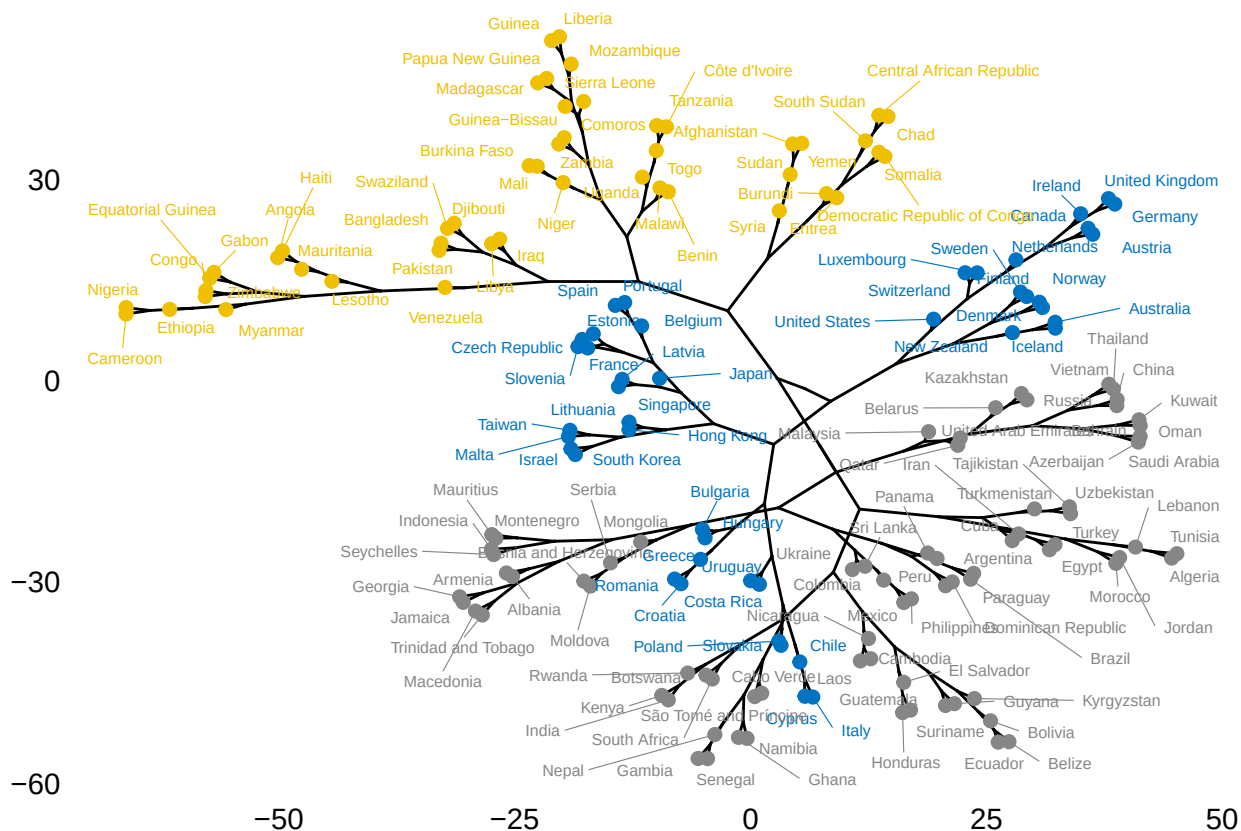
Apply Hierarchical Clustering with Different Linkage Methods and visualize Dendrograms for each Linkage Method

Ward's method

```
rownames(country) <- country[, 1]
dd <- dist(scale(country[, -1]), method = "euclidean")
```

```
hc_ward <- hclust(dd, method = "ward.D2")

fviz_dend(hc_ward,
  cex = 0.5,
  main = "Dendrogram - Ward's Method",
  k = 3,
  k_colors = "jco",
  type = "phylogenetic",
  repel = TRUE)
```



```
# OR
png("dendrogram_plot_ward.png", width = 1200, height = 800)
fviz_dend(hc_ward,
  cex = 0.5,
  main = "Dendrogram - Ward's Method",
  k = 3,
  k_colors = c("red", "blue", "darkgreen"))
```

```
## Warning: The 'scale' argument of 'guides()' cannot be 'FALSE'. Use "none" instead as
## of ggplot2 3.3.4.
## i The deprecated feature was likely used in the factoextra package.
## Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

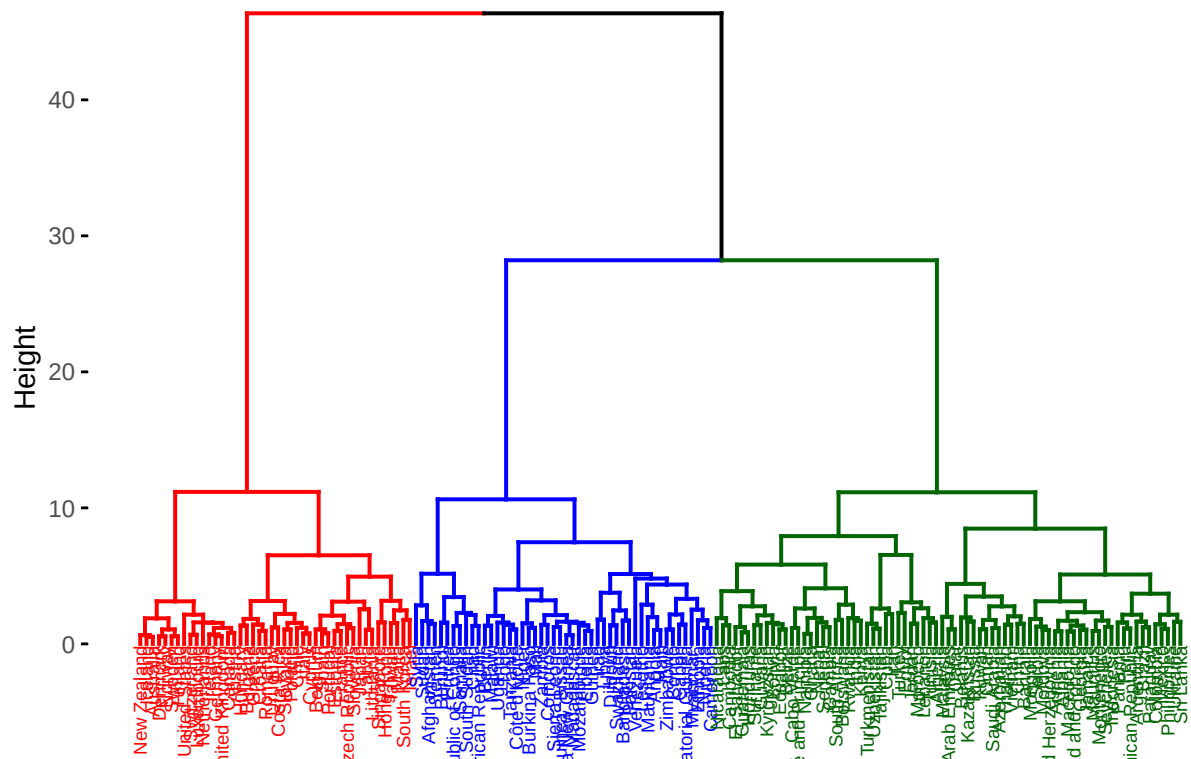
```
dev.off()
```

```
## pdf
## 2
```

```
fviz_dend(hc_ward,
  cex = 0.5,
  main = "Dendrogram - Ward's Method",
```

```
k = 3,  
k_colors = c("red", "blue", "darkgreen"))
```

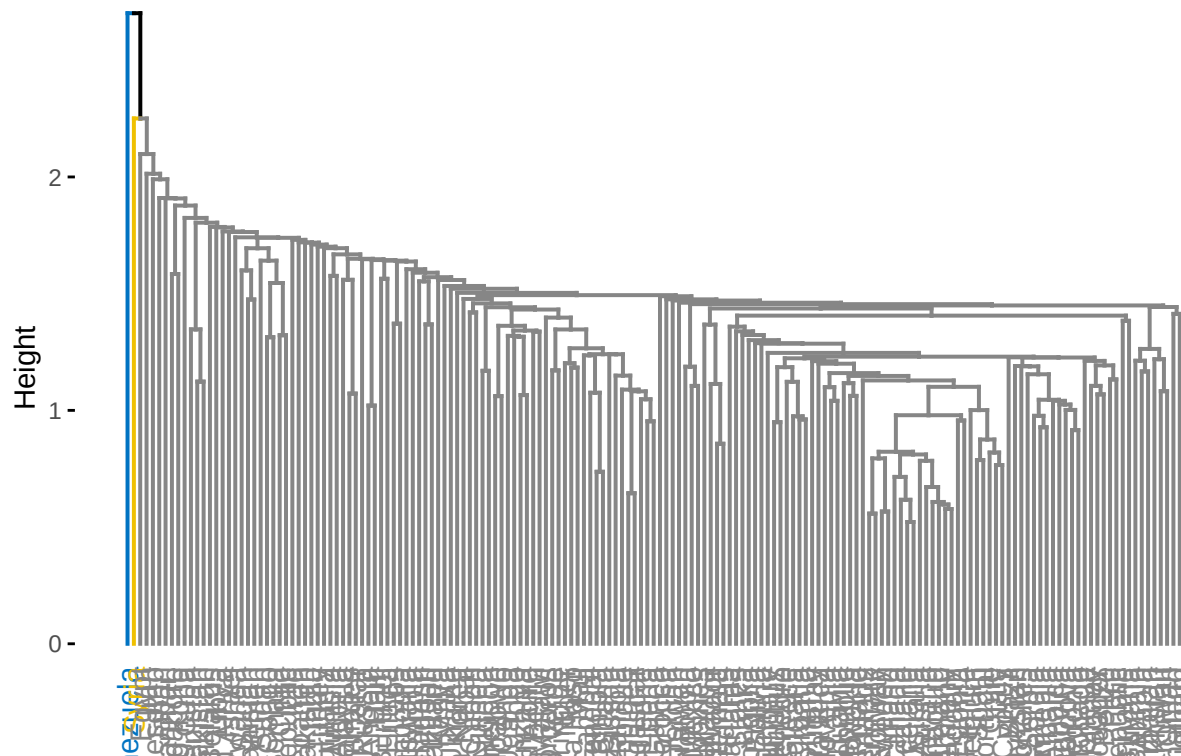
Dendrogram – Ward's Method



Single linkage

```
# Hierarchical clustering for Single Linkage  
hc_single <- hclust(dd, method = "single")  
  
# Display Single Linkage Dendrogram  
fviz_dend(hc_single,  
  main = "Dendrogram - Single Linkage",  
  k = 3,  
  k_colors = "jco")
```

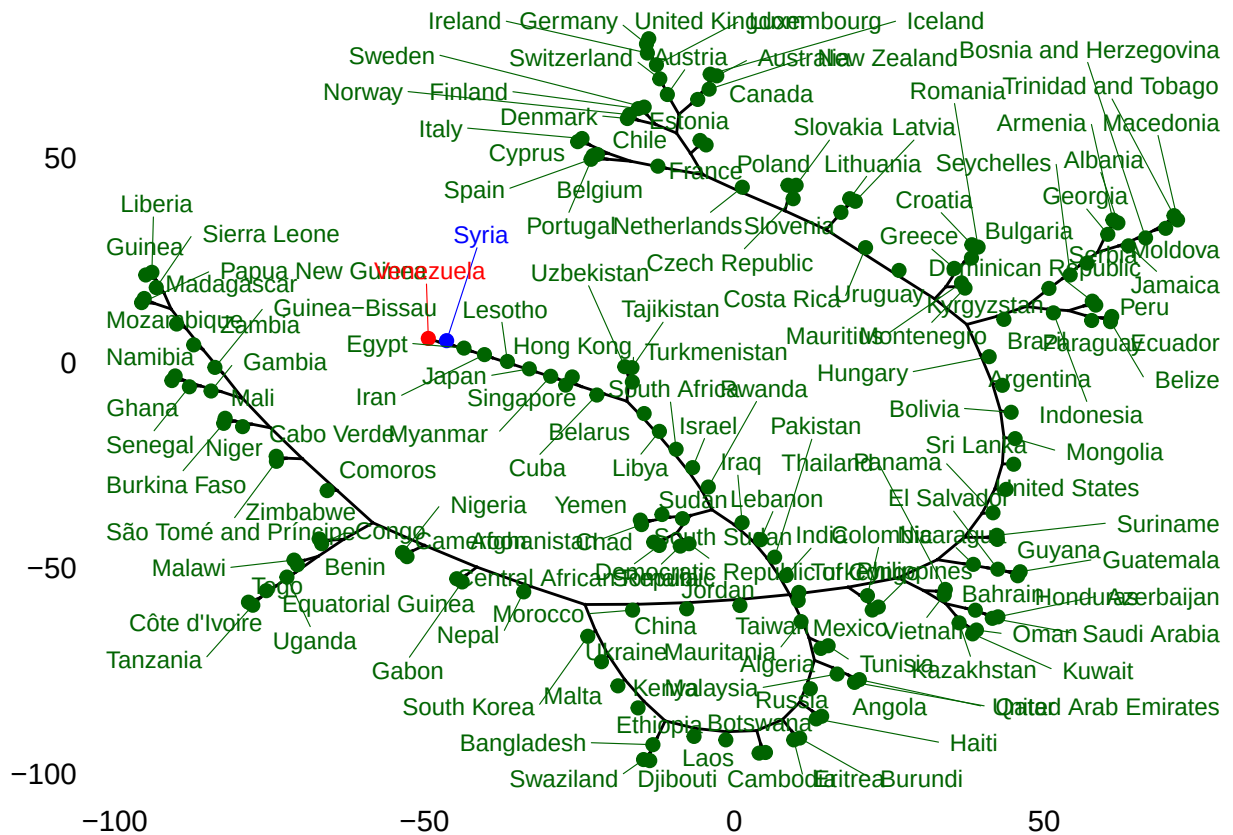
Dendrogram – Single Linkage



```
# Save the Single Linkage Dendrogram
png("dendrogram_plot_single.png", width = 1200, height = 800)
fviz_dend(hc_single,
  k = 3,
  k_colors = c("red", "blue", "darkgreen"),
  type = "phylogenetic",
  repel = TRUE)
dev.off()
```

```
## pdf
## 2
```

```
# Display Single Linkage Dendrogram
fviz_dend(hc_single,
  k = 3,
  k_colors = c("red", "blue", "darkgreen"),
  type = "phylogenetic",
  repel = TRUE)
```

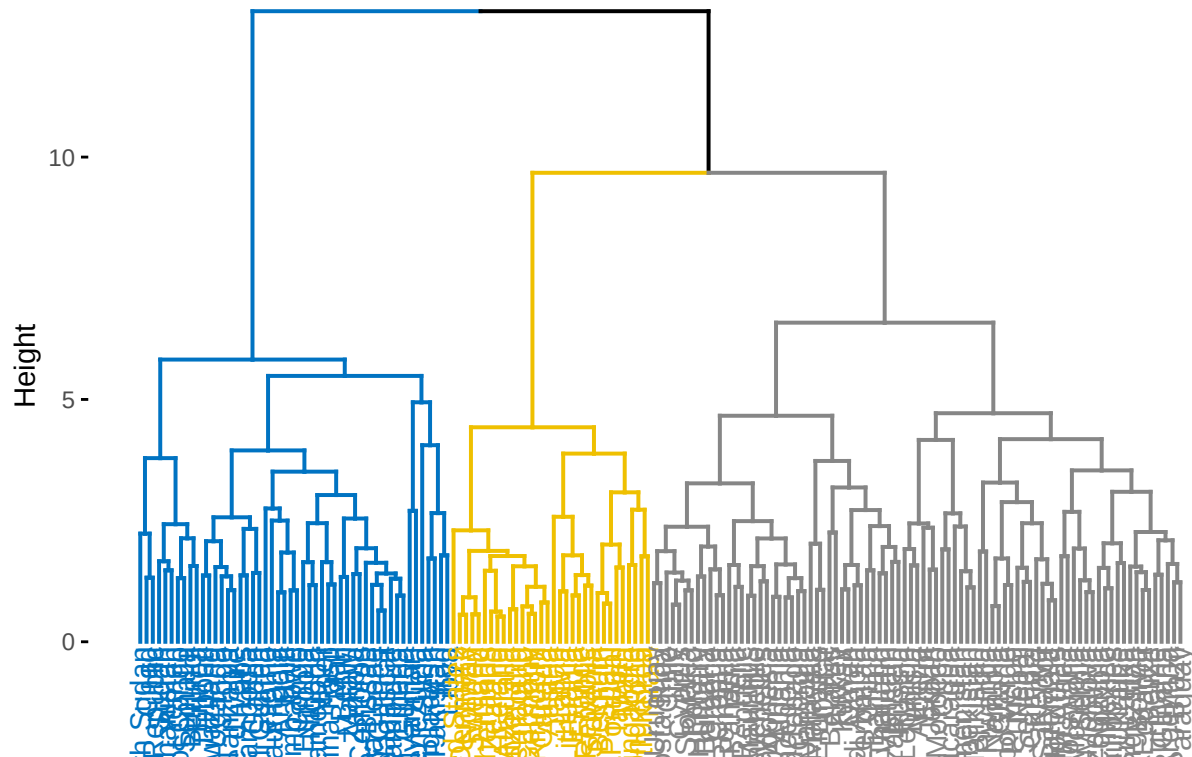


Complete linkage

```
# Hierarchical clustering for Complete Linkage
hc_complete <- hclust(dd, method = "complete")

# Display Complete Linkage Dendrogram (circular type)
fviz_dend(hc_complete,
  main = "Dendrogram - Complete Linkage",
  k = 3,
  k_colors = "jco")
```

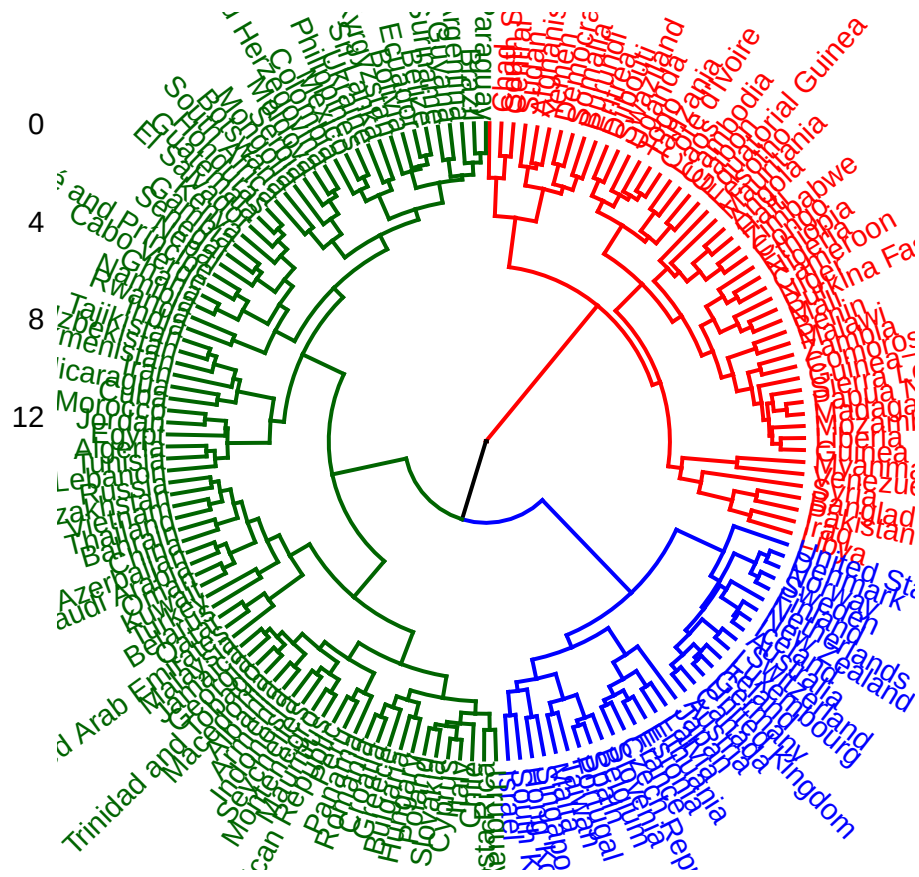
Dendrogram – Complete Linkage



```
# Save the circular Complete Linkage Dendrogram to an image
png("dendrogram_plot_complete_circular.png", width = 1200, height = 800)
fviz_dend(hc_complete,
  main = "Dendrogram - Complete Linkage",
  k = 3,
  k_colors = c("red", "blue", "darkgreen"),
  type = "circular")
dev.off()
```

```
## pdf
## 2
```

```
# Display the circular Complete Linkage Dendrogram
fviz_dend(hc_complete,
  main = "Dendrogram - Complete Linkage",
  k = 3,
  k_colors = c("red", "blue", "darkgreen"),
  type = "circular")
```



Average linkage

```
# Hierarchical clustering for Average Linkage
hc_average <- hclust(dd, method = "average")

# Save the Average Linkage Dendrogram to an image
png("dendrogram_plot_average_linkage.png", width = 1200, height = 800)
fviz_dend(hc_average,
  main = "Dendrogram - Average Linkage",
  k = 3,
  k_colors = c("red", "blue", "darkgreen"),
  cex = 0.5,
  rotate = FALSE,
  mar = c(3, 5, 2, 1))
dev.off()
```

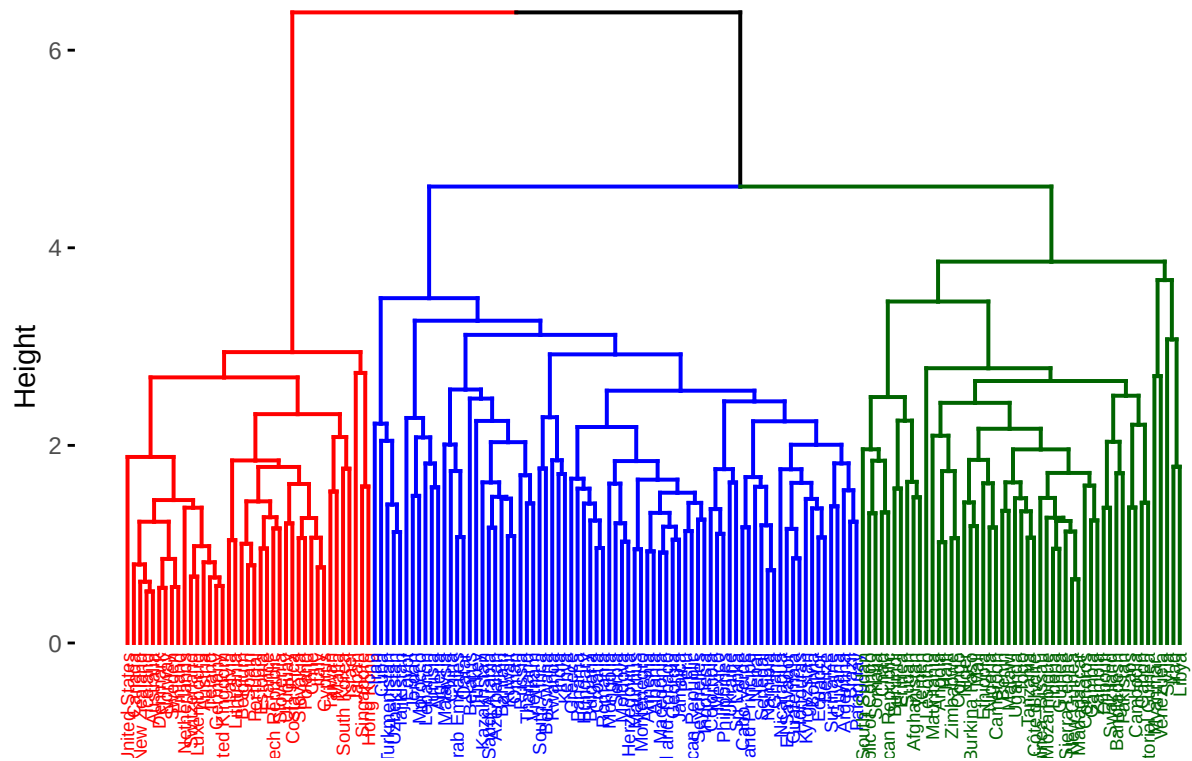
```
## pdf
## 2
```

```
# Display the Average Linkage Dendrogram
fviz_dend(hc_average,
  main = "Dendrogram - Average Linkage",
  k = 3,
```

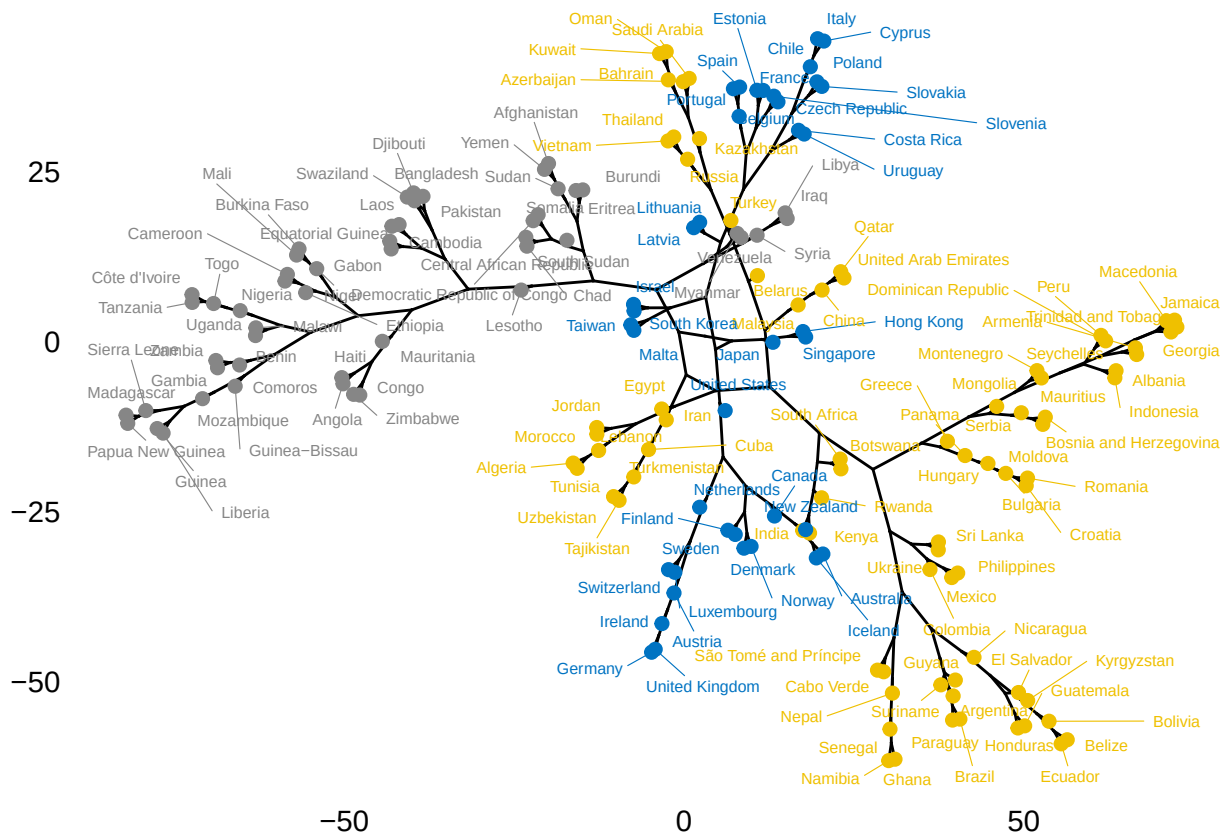


```
k_colors = c("red", "blue", "darkgreen"),
cex = 0.5,
rotate = FALSE,
mar = c(3, 5, 2, 1))
```

Dendrogram – Average Linkage



```
fviz_dend(hc_average, cex = 0.5, main = "Dendrogram - Ward's Method", k = 3, k_colors = "jco", type = "j")
```

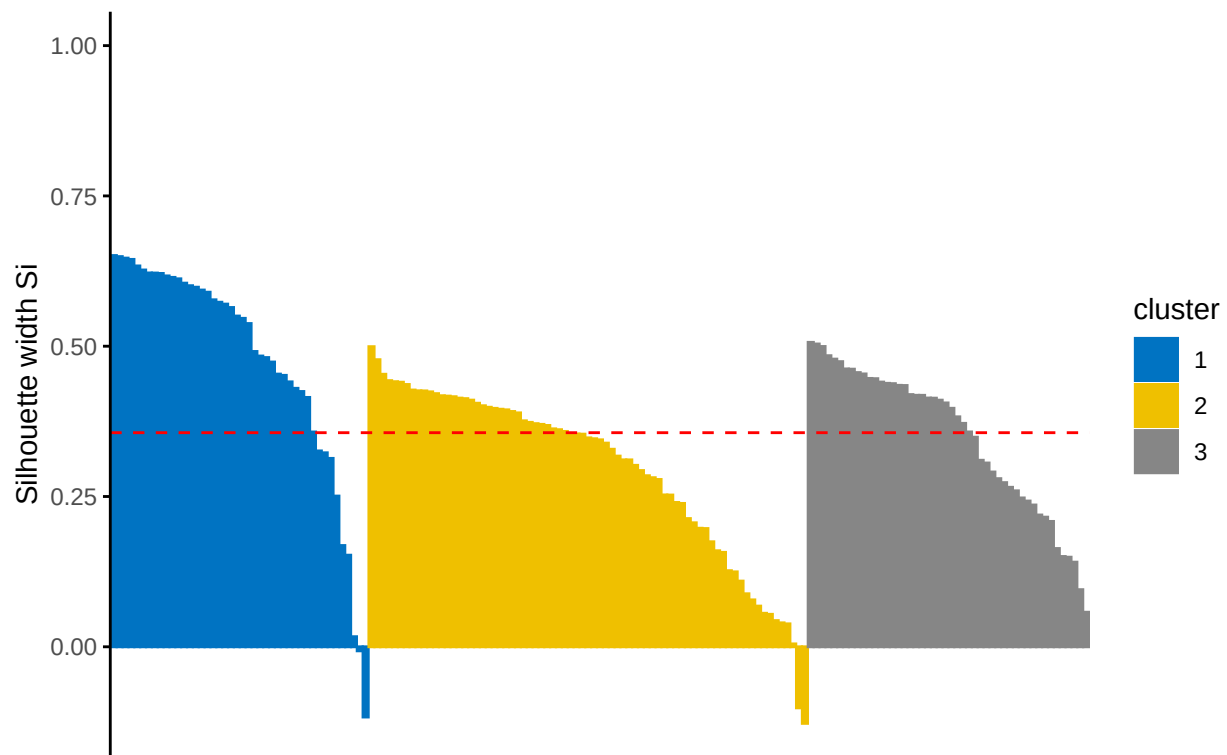


Cluster Validity Indices (Silhouette Score)

```
fviz_silhouette(hc.res, palette = "jco", ggtheme = theme_classic()) +
  labs(title = "Silhouette Analysis - Hierarchical Clustering")
```

##	cluster	size	ave.sil.width
## 1	1	44	0.47
## 2	2	75	0.29
## 3	3	48	0.35

Silhouette Analysis – Hierarchical Clustering



```
# Silhouette information
silinfo <- hc.res$silinfo
sil <- silinfo$widths # Extract silhouette widths
# Average silhouette width of each cluster
cat("Cluster-wise average silhouette widths:\n")
```

```
## Cluster-wise average silhouette widths:
```

```
print(silinfo$clus.avg.widths)
```

```
## [1] 0.4726346 0.2926953 0.3484828
```

```
# The total average (mean of all individual silhouette widths)
cat("\nTotal average silhouette width:\n")
```

```
##
## Total average silhouette width:
```

```
print(silinfo$avg.width)
```

```
## [1] 0.3561392
```

```
# The size of each clusters
cat("\nCluster sizes:\n")
```

```
##
## Cluster sizes:
```

```
print(hc.res$size)
```

```
## [1] 44 75 48
```

We can see that average width in Cluster 1 is greater than the other averages, assuming a better clustering in these observations, and the average Silhouette coefficient with is greater than 0 (0.3561392) so we could assume that the observations are clustered, however there might be some overlapping, we have to check if there are samples with a negative coefficient.

```
# Silhouette widths of each observation
silinfo$widths[, 1:3]
```

```
##      cluster neighbor    sil_width
## 12         1         2 0.650874451
## 14         1         2 0.649081061
## 11         1         2 0.646599142
## 9          1         2 0.644423116
## 13         1         2 0.633588074
## 7          1         2 0.626806576
## 8          1         2 0.621922097
## 18         1         2 0.621786850
## 21         1         2 0.621053714
## 23         1         2 0.616981155
## 15         1         2 0.614518464
## 5          1         2 0.612149855
## 6          1         2 0.604960962
## 3          1         2 0.600705128
## 10         1         2 0.598210229
## 4          1         2 0.593451511
## 2          1         2 0.589897845
## 1          1         2 0.577222065
## 24         1         2 0.573187108
## 25         1         2 0.570103435
## 26         1         2 0.564402850
## 19         1         2 0.550223654
## 27         1         2 0.546419241
## 20         1         2 0.537748984
## 30         1         2 0.491252141
## 22         1         2 0.483739440
## 16         1         2 0.481154163
## 31         1         2 0.473749498
## 34         1         2 0.453633808
## 28         1         2 0.451556627
## 17         1         2 0.440737461
## 29         1         2 0.429992409
## 32         1         2 0.424676225
```

## 35	1	2	0.414826887
## 36	1	2	0.357358524
## 37	1	2	0.325847459
## 33	1	2	0.322751011
## 39	1	2	0.313444796
## 38	1	2	0.250742696
## 41	1	2	0.168468719
## 40	1	2	0.152436128
## 42	1	2	0.016830471
## 45	1	2	-0.006764587
## 48	1	2	-0.116830143
## 70	2	3	0.499249618
## 72	2	3	0.477612087
## 74	2	3	0.453381520
## 69	2	1	0.442803591
## 68	2	1	0.441025991
## 85	2	3	0.440175156
## 76	2	3	0.436373323
## 73	2	1	0.427154075
## 79	2	1	0.425863341
## 61	2	1	0.425708035
## 94	2	3	0.424123579
## 65	2	1	0.421124957
## 60	2	1	0.417666971
## 91	2	3	0.417065475
## 67	2	1	0.416030156
## 77	2	1	0.413716644
## 84	2	3	0.412844771
## 92	2	3	0.410377161
## 63	2	1	0.405205417
## 97	2	3	0.401013719
## 55	2	1	0.398494607
## 56	2	1	0.396673981
## 71	2	1	0.395228014
## 81	2	3	0.394222159
## 78	2	1	0.391452590
## 57	2	1	0.389169155
## 86	2	3	0.375944528
## 53	2	1	0.373360073
## 88	2	3	0.371218003
## 64	2	1	0.370017992
## 95	2	3	0.368230613
## 89	2	3	0.362566569
## 80	2	3	0.361158223
## 52	2	1	0.358263060
## 87	2	3	0.354564068
## 62	2	1	0.354008358
## 75	2	3	0.353773971
## 58	2	1	0.347575347
## 93	2	3	0.346117211
## 59	2	1	0.344327778
## 66	2	1	0.338851133
## 83	2	3	0.328663225
## 100	2	3	0.317249853

## 101	2	3	0.311006629
## 96	2	3	0.310967008
## 82	2	3	0.301886982
## 99	2	3	0.293199206
## 105	2	3	0.284615694
## 104	2	3	0.281335581
## 90	2	3	0.278349572
## 54	2	1	0.252646481
## 51	2	1	0.252505573
## 107	2	3	0.240091690
## 103	2	3	0.238494545
## 98	2	3	0.213321331
## 109	2	3	0.206307745
## 113	2	3	0.197249777
## 106	2	3	0.196749862
## 49	2	1	0.174709331
## 102	2	3	0.159756229
## 108	2	3	0.157137859
## 47	2	1	0.126642338
## 114	2	3	0.124709887
## 112	2	3	0.109335674
## 50	2	1	0.088210922
## 115	2	3	0.078082315
## 110	2	3	0.067728710
## 111	2	3	0.055796081
## 126	2	3	0.054076990
## 121	2	3	0.043617390
## 118	2	3	0.039886187
## 46	2	1	0.038211584
## 44	2	1	0.004841383
## 43	2	1	-0.101584123
## 116	2	3	-0.127385498
## 163	3	2	0.506425745
## 161	3	2	0.503726044
## 162	3	2	0.499560716
## 156	3	2	0.484263107
## 155	3	2	0.478565915
## 157	3	2	0.474360203
## 150	3	2	0.462237588
## 160	3	2	0.461841698
## 165	3	2	0.456192552
## 164	3	2	0.453907620
## 166	3	2	0.446463077
## 148	3	2	0.446028211
## 144	3	2	0.440403946
## 151	3	2	0.438295688
## 167	3	2	0.437754466
## 153	3	2	0.434978097
## 152	3	2	0.434673145
## 149	3	2	0.419585006
## 147	3	2	0.418378179
## 137	3	2	0.418340336
## 141	3	2	0.413880999
## 158	3	2	0.413647546

```
## 142      3      2 0.410319304
## 138      3      2 0.405710702
## 133      3      2 0.396796015
## 135      3      2 0.382596382
## 154      3      2 0.371707036
## 139      3      2 0.357444298
## 130      3      2 0.348693815
## 132      3      2 0.310345764
## 134      3      2 0.305756029
## 127      3      2 0.290798057
## 159      3      2 0.279707000
## 129      3      2 0.273108560
## 128      3      2 0.265341672
## 125      3      2 0.259506097
## 120      3      2 0.247651956
## 123      3      2 0.242493357
## 131      3      2 0.235914635
## 143      3      2 0.219270222
## 136      3      2 0.215695835
## 146      3      2 0.208556540
## 117      3      2 0.163522108
## 119      3      2 0.150221260
## 140      3      2 0.149066448
## 145      3      2 0.140856528
## 122      3      2 0.095043572
## 124      3      2 0.057539701
```

```
# Identify Negative Silhouette Observations
neg_sil_index <- which(sil[, "sil_width"] < 0)
neg_sil_samples <- sil[neg_sil_index, , drop = FALSE]

cat("\nNegative silhouette samples:\n")
```

```
##
## Negative silhouette samples:
```

```
print(neg_sil_samples)
```

```
##      cluster neighbor    sil_width
## 45         1         2 -0.006764587
## 48         1         2 -0.116830143
## 43         2         1 -0.101584123
## 116        2         3 -0.127385498
```

```
cat("\nNumber of negative silhouette samples:", nrow(neg_sil_samples))
```

```
##
## Number of negative silhouette samples: 4
```

We can see that samples 45 & 48 in Cluster 1 and samples 43 & 116 in Cluster 2, have a negative silhouette coefficient, meaning that they are not in the right cluster (or could be considered as outliers), and they should belong to Cluster 2, Cluster 2, Cluster 1 and Cluster 3 respectively.

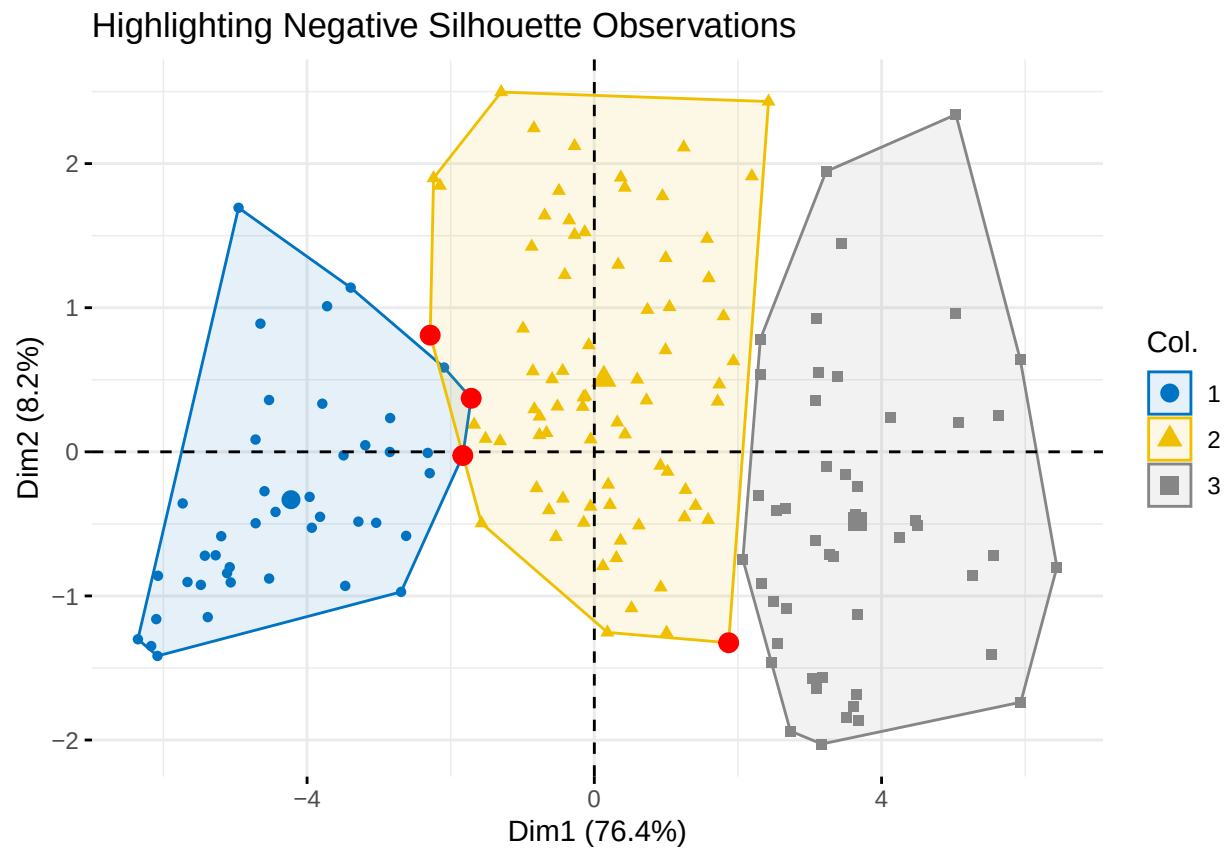
```

# Convert clusters to a factor
hc_clust <- as.factor(hc.res$cluster)

# Perform PCA
pca_result <- prcomp(country_scaled)

# PCA Plot with Negative Silhouette Samples Highlighted
fviz_pca_ind(
  pca_result,
  geom = "point",
  ellipse.type = "convex",
  col.ind = hc_clust, # Use factor for discrete palette
  palette = "jco",
  addEllipses = TRUE,
  ggtheme = theme_minimal()
) +
  geom_point(
    data = as.data.frame(pca_result$x)[rownames(neg_sil_samples), ],
    aes(x = PC1, y = PC2),
    color = "red", size = 3
  ) +
  labs(title = "Highlighting Negative Silhouette Observations")

```



Cluster Stability Validation

For the Cluster Stability Validation the following indices are going to be evaluated to assess the quality of clustering results.

APN (Average Proportion of Non-overlap) Description: Measures the extent to which clustering assignments change when a subset of the data is removed. Lower values indicate more stable clustering solutions. Purpose: Evaluates the stability of clusters under resampling.

AD (Average Distance) Description: Measures the average distance between observations and the centroids of their assigned clusters. Purpose: Evaluates the compactness of the clusters.

ADM (Average Distance between Means) Description: Measures the average distance between cluster centroids (means). Purpose: Evaluates the separation between clusters.

FOM (Figure of Merit) Description: Measures the prediction error when clustering is applied to subsets of the data. This is computed by predicting the cluster membership of the left-out subset using the clusters formed on the remaining data. Purpose: Evaluates the predictive performance of the clustering model.

```
clust_validation <- clValid(obj = country_scaled, nClust = 3, clMethods = "hierarchical", validation =
  maxitems = 600, metric = "euclidean", method = "ward"
)
```

```
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
## The "ward" method has been renamed to "ward.D"; note new "ward.D2"
```

```
## Warning in clValid(obj = country_scaled, nClust = 3, clMethods =
## "hierarchical", : rownames for data not specified, using 1:nrow(data)
```

```
# Print summary of the stability validation results
summary(clust_validation)
```

```
##
## Clustering Methods:
## hierarchical
##
## Cluster sizes:
## 3
##
## Validation Measures:
## 3
##
## hierarchical APN 0.1014
```

```
##          AD    2.8490
##          ADM   0.4077
##          FOM   0.5864
```

```
##
```

```
## Optimal Scores:
```

```
##
```

```
##      Score Method      Clusters
## APN 0.1014 hierarchical 3
## AD  2.8490 hierarchical 3
## ADM 0.4077 hierarchical 3
## FOM 0.5864 hierarchical 3
```

```
clust_validation <- clValid(obj = country_scaled, nClust = 3, clMethods = "hierarchical", validation =
  maxitems = 600, metric = "euclidean", method = "single"
)
```

```
## Warning in clValid(obj = country_scaled, nClust = 3, clMethods =
## "hierarchical", : rownames for data not specified, using 1:nrow(data)
```

```
# Print summary of the stability validation results
summary(clust_validation)
```

```
##
```

```
## Clustering Methods:
```

```
## hierarchical
```

```
##
```

```
## Cluster sizes:
```

```
## 3
```

```
##
```

```
## Validation Measures:
```

```
##          3
```

```
##
```

```
## hierarchical APN 0.0009
```

```
##          AD  4.4786
```

```
##          ADM 0.0070
```

```
##          FOM 0.9941
```

```
##
```

```
## Optimal Scores:
```

```
##
```

```
##      Score Method      Clusters
## APN 0.0009 hierarchical 3
## AD  4.4786 hierarchical 3
## ADM 0.0070 hierarchical 3
## FOM 0.9941 hierarchical 3
```

```
clust_validation <- clValid(obj = country_scaled, nClust = 3, clMethods = "hierarchical", validation =
  maxitems = 600, metric = "euclidean", method = "complete"
)
```

```
## Warning in clValid(obj = country_scaled, nClust = 3, clMethods =
## "hierarchical", : rownames for data not specified, using 1:nrow(data)
```

```
# Print summary of the stability validation results
summary(clust_validation)
```

```
##
## Clustering Methods:
## hierarchical
##
## Cluster sizes:
## 3
##
## Validation Measures:
## 3
##
## hierarchical APN 0.1445
## AD 2.9810
## ADM 0.7243
## FOM 0.5996
##
## Optimal Scores:
##
## Score Method Clusters
## APN 0.1445 hierarchical 3
## AD 2.9810 hierarchical 3
## ADM 0.7243 hierarchical 3
## FOM 0.5996 hierarchical 3
```

```
clust_validation <- clValid(obj = country_scaled, nClust = 3, clMethods = "hierarchical", validation =
  maxitems = 600, metric = "euclidean", method = "average"
)
```

```
## Warning in clValid(obj = country_scaled, nClust = 3, clMethods =
## "hierarchical", : rownames for data not specified, using 1:nrow(data)
```

```
# Print summary of the stability validation results
summary(clust_validation)
```

```
##
## Clustering Methods:
## hierarchical
##
## Cluster sizes:
## 3
##
## Validation Measures:
## 3
##
## hierarchical APN 0.0705
## AD 2.7748
## ADM 0.3060
## FOM 0.5773
##
## Optimal Scores:
```

```
##
##      Score  Method      Clusters
## APN 0.0705 hierarchical 3
## AD  2.7748 hierarchical 3
## ADM 0.3060 hierarchical 3
## FOM 0.5773 hierarchical 3
```

The cophenetic correlation coefficient measures how well the hierarchical clustering preserves the pairwise distances between points. So we can infer that the linkage method with the higher cophenetic correlation coefficient best preserves the original distances.

```
# List of linkage methods to compare
linkage_methods <- c("ward.D2", "single", "complete", "average")

# Initialize an empty data frame to store cophenetic correlation results
cophenetic_results <- data.frame(Method = character(), Cophenetic_Correlation = numeric())

# Compute cophenetic correlation for each linkage method
for (method in linkage_methods) {
  # Perform hierarchical clustering with the specified method
  hc <- hclust(diss_matrix, method = method)

  # Calculate cophenetic correlation
  cor <- cor(cophenetic(hc), as.dist(diss_matrix))

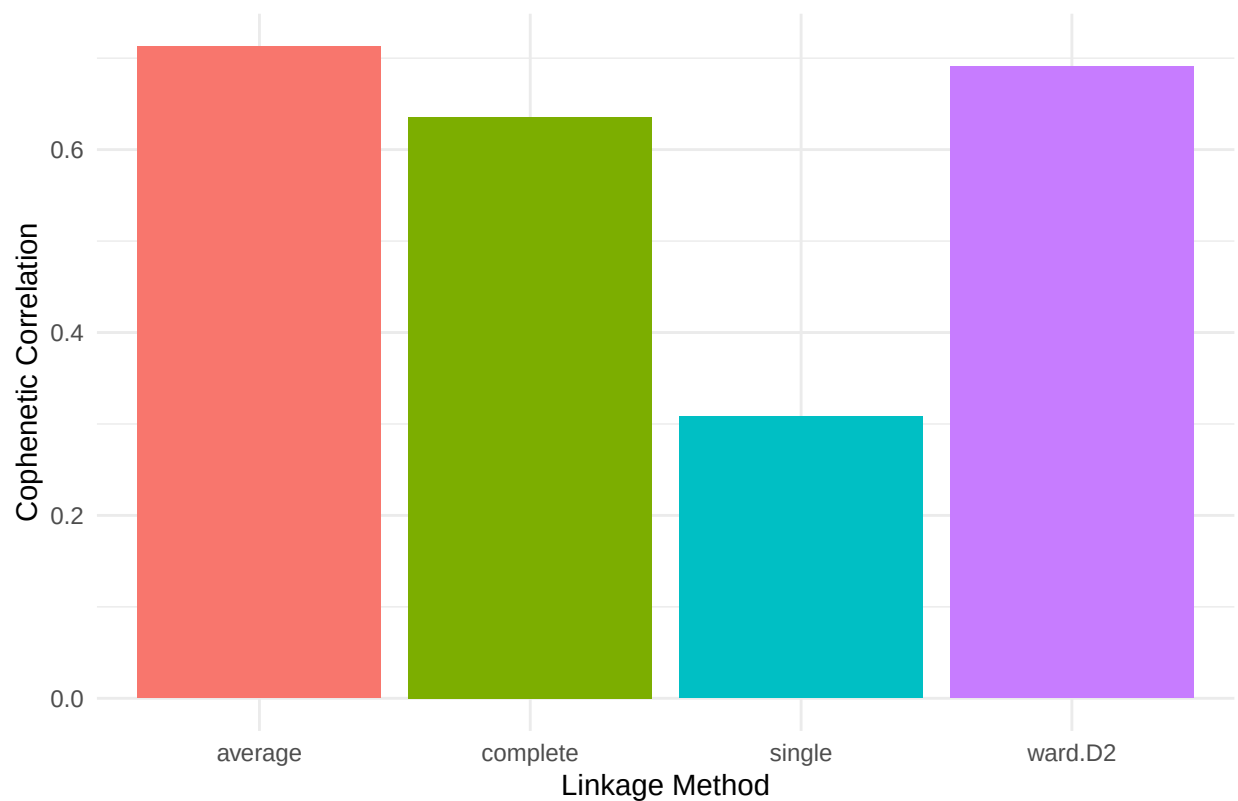
  # Append the results to the data frame
  cophenetic_results <- rbind(cophenetic_results,
                             data.frame(Method = method, Cophenetic_Correlation = cor))
}

# Print cophenetic correlation results
print(cophenetic_results)
```

```
##      Method Cophenetic_Correlation
## 1 ward.D2          0.6910012
## 2 single          0.3080855
## 3 complete        0.6354667
## 4 average          0.7129493
```

```
# Bar plot of cophenetic correlations
ggplot(cophenetic_results, aes(x = Method, y = Cophenetic_Correlation, fill = Method)) +
  geom_bar(stat = "identity", show.legend = FALSE) +
  labs(title = "Comparison of Cophenetic Correlation Coefficients",
       x = "Linkage Method", y = "Cophenetic Correlation") +
  theme_minimal()
```

Comparison of Cophenetic Correlation Coefficients



Comparing Hierarchical Clustering vs other methods

```
# Stability measures
clmethods <- c("hierarchical","kmeans","pam")
stab <- clValid(country_scaled, nClust = 2:10, clMethods = clmethods,
                validation = "stability")
```

```
## Warning in clValid(country_scaled, nClust = 2:10, clMethods = clmethods, :
## rownames for data not specified, using 1:nrow(data)
```

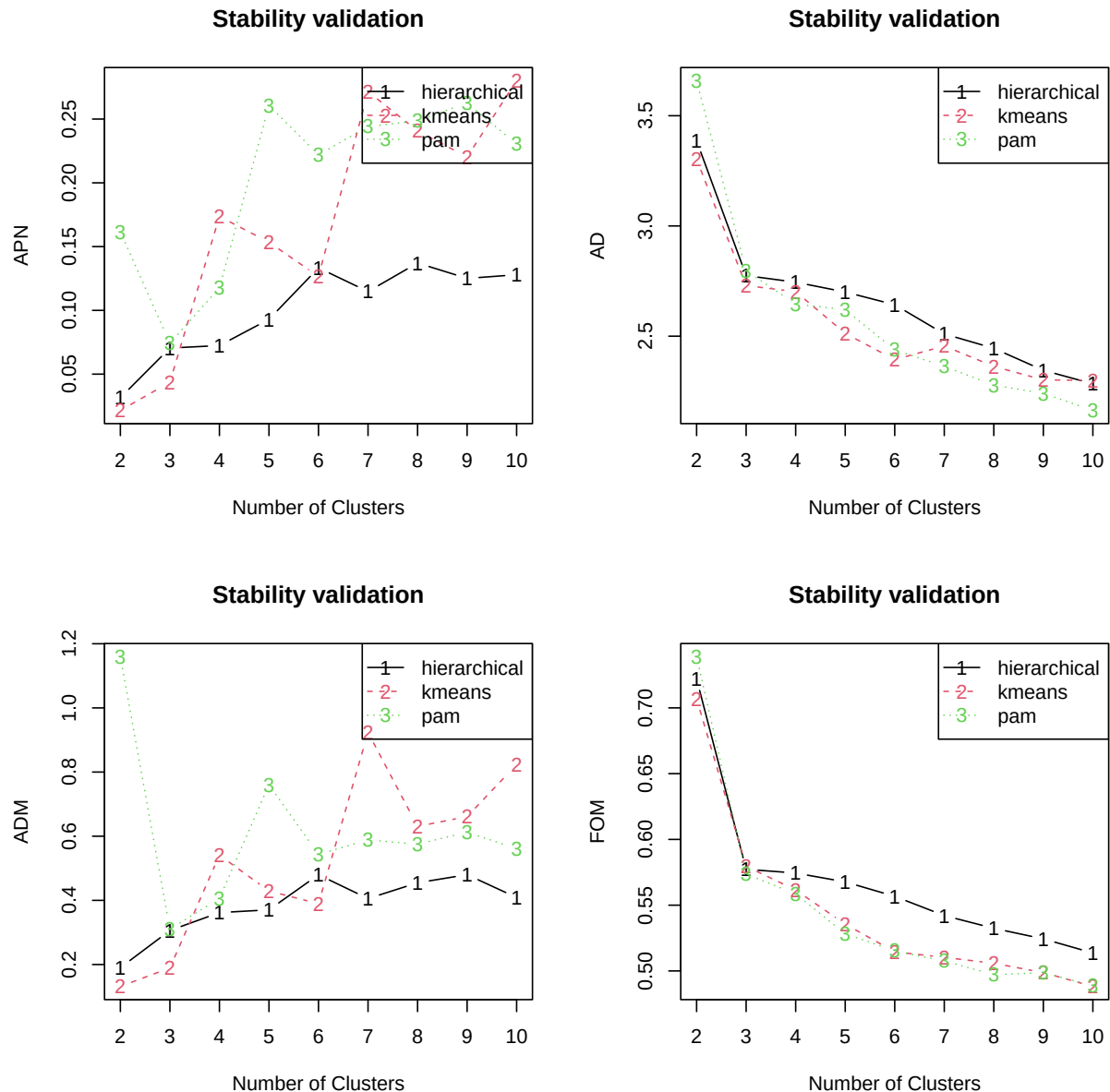
```
# Display only optimal Scores
summary(stab)
```

```
##
## Clustering Methods:
## hierarchical kmeans pam
##
## Cluster sizes:
## 2 3 4 5 6 7 8 9 10
##
## Validation Measures:
##           2       3       4       5       6       7       8       9       10
##
```

```
## hierarchical APN 0.0315 0.0705 0.0724 0.0930 0.1333 0.1148 0.1371 0.1251 0.1280
##              AD 3.3864 2.7748 2.7449 2.7004 2.6426 2.5101 2.4448 2.3451 2.2837
##              ADM 0.1901 0.3060 0.3619 0.3705 0.4798 0.4056 0.4548 0.4800 0.4089
##              FOM 0.7221 0.5773 0.5744 0.5677 0.5567 0.5421 0.5324 0.5244 0.5137
## kmeans      APN 0.0215 0.0436 0.1737 0.1534 0.1266 0.2715 0.2408 0.2203 0.2801
##              AD 3.3029 2.7294 2.7021 2.5129 2.3937 2.4556 2.3629 2.3016 2.2994
##              ADM 0.1317 0.1893 0.5409 0.4293 0.3896 0.9237 0.6307 0.6611 0.8234
##              FOM 0.7069 0.5796 0.5615 0.5356 0.5145 0.5103 0.5058 0.4988 0.4882
## pam         APN 0.1616 0.0743 0.1176 0.2606 0.2220 0.2441 0.2486 0.2627 0.2304
##              AD 3.6588 2.7966 2.6458 2.6203 2.4378 2.3630 2.2780 2.2404 2.1631
##              ADM 1.1596 0.3103 0.4062 0.7605 0.5433 0.5886 0.5750 0.6126 0.5610
##              FOM 0.7389 0.5737 0.5585 0.5283 0.5153 0.5076 0.4972 0.4987 0.4889
##
## Optimal Scores:
##
##      Score Method Clusters
## APN 0.0215 kmeans 2
## AD 2.1631 pam 10
## ADM 0.1317 kmeans 2
## FOM 0.4882 kmeans 10
```

```
par(mfrow = c(2, 2))

# Plot the stability measures
plot(stab)
```



```
par(mfrow = c(1, 1))
```

In APN Validation we can see that the lowest value comes with K-means Method and 2 Clusters In AD Validation we can see that the lowest value comes with PAM Method and 10 Clusters In ADM Validation we can see that the Highest value comes with K-means Method and 2 Clusters In FOM Validation we can see that the lowest value comes with K-means Method and 10 Clusters

```
# Stability Measures Across Clustering Methods
stab_scores <- as.data.frame(as.table(stab@measures))

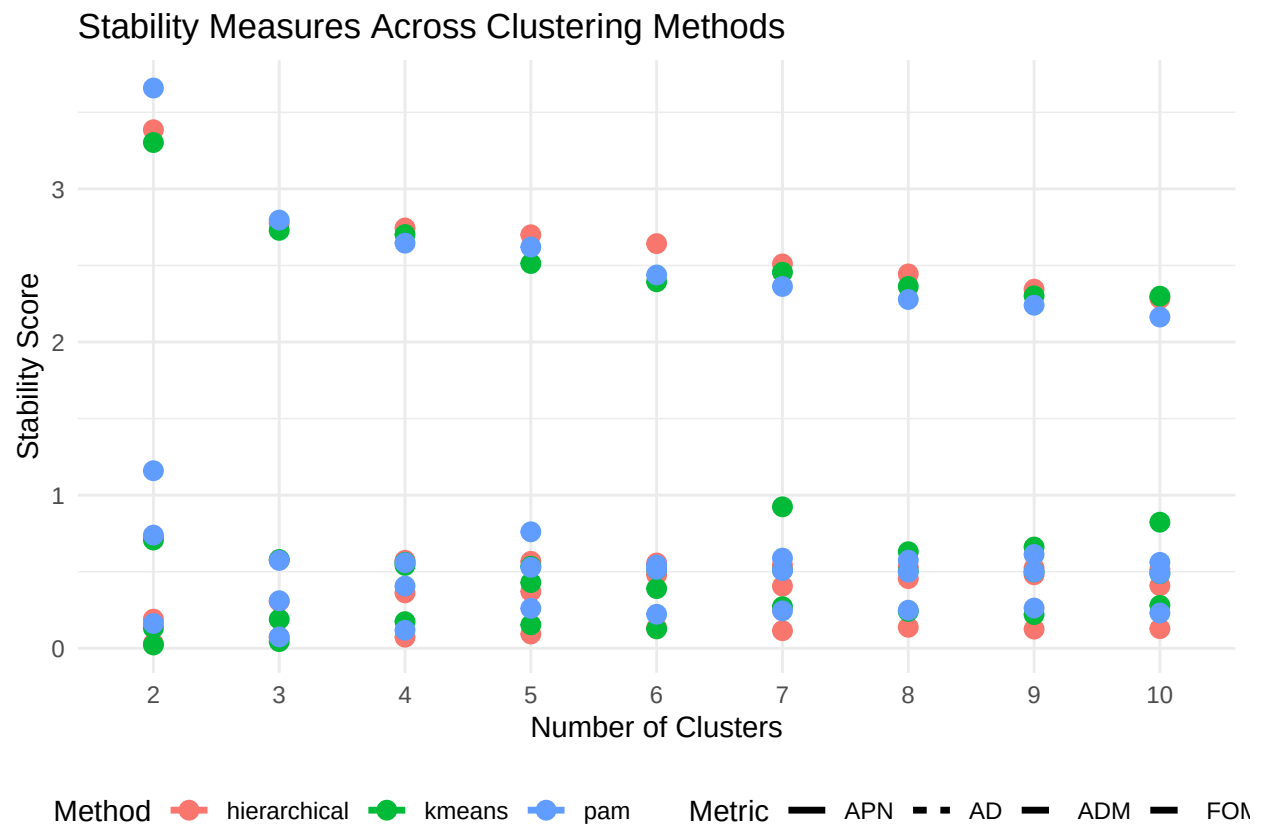
colnames(stab_scores) <- c("Metric", "Clusters", "Method", "Stability_Score")

# Plot stability measures
```

```
ggplot(stab_scores, aes(x = Clusters, y = Stability_Score, color = Method, linetype = Metric)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  labs(title = "Stability Measures Across Clustering Methods",
       x = "Number of Clusters", y = "Stability Score") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
## 'geom_line()': Each group consists of only one observation.
## i Do you need to adjust the group aesthetic?
```



```
# Dunn Index for the Hierarchical clustering

dunn_index <- cluster.stats(diss_matrix, hc.res$cluster)$dunn
print(paste("Dunn Index:", dunn_index))
```

```
## [1] "Dunn Index: 0.203546895548816"
```



```

# Extract components of Dunn Index
dunn_components <- cluster.stats(diss_matrix, hc.res$cluster)
min_inter_dist <- dunn_components$min.separation
min_inter_dist

## [1] 1.185192

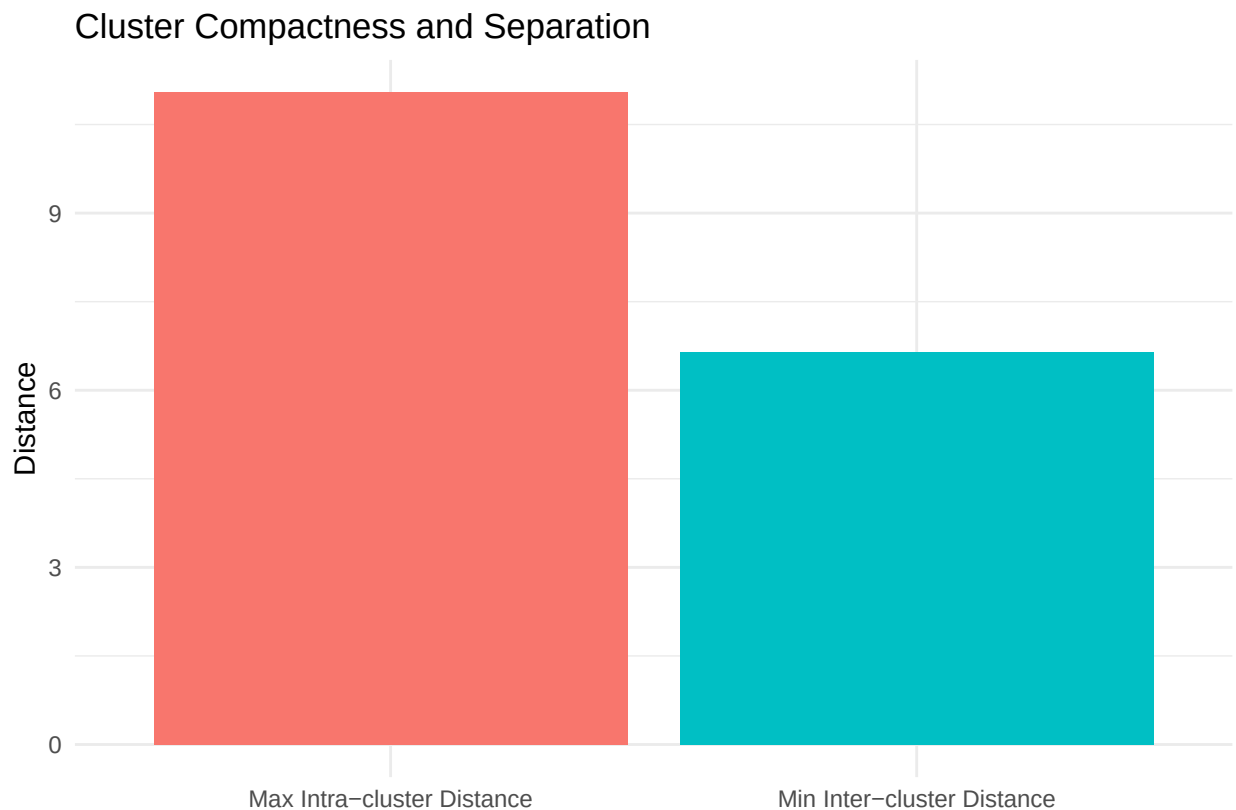
max_intra_dist <- dunn_components$diameter
max_intra_dist

## [1] 5.218879 5.454060 5.822696

# Plot compactness vs separation
compactness_separation <- data.frame(
  Metric = c("Max Intra-cluster Distance", "Min Inter-cluster Distance"),
  Value = c(max_intra_dist, min_inter_dist)
)

ggplot(compactness_separation, aes(x = Metric, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", show.legend = FALSE) +
  labs(title = "Cluster Compactness and Separation", x = "", y = "Distance") +
  theme_minimal()

```



The Dunn Index value of 0.2035 indicates that the the separation between the clusters formed by the Hierarchical method may not be very strong, as it is closer to 0 than to 1.

Cluster Analysis

In this section, we are going to demonstrate how to apply average linkage to further analyze and define the clusters.

Cut the Dendrogram into Clusters

```
# Cut the tree into 3 clusters
clusters <- cutree(hc_average, k = 3)

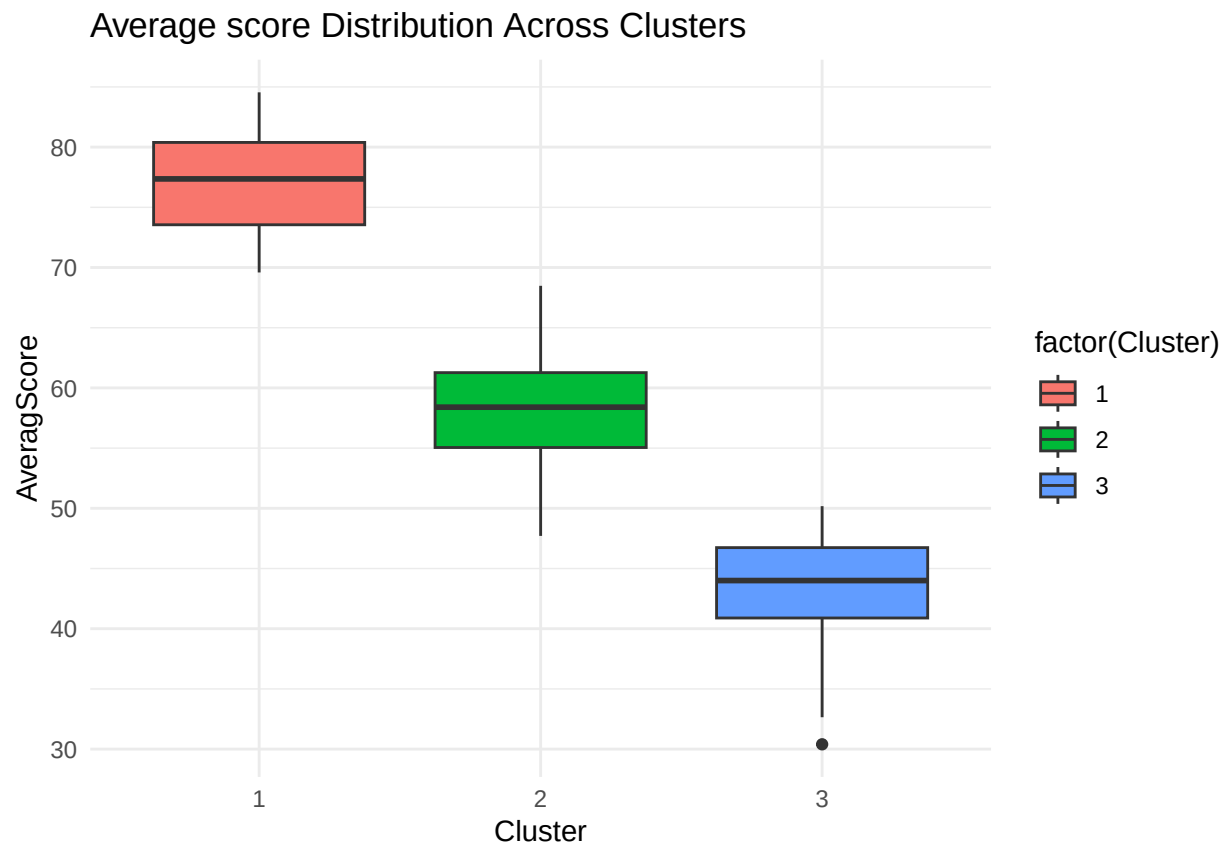
# Add the cluster assignments back to your original data
country$Cluster <- clusters
```

Group the Data by Cluster

```
country1 <- country[, -1]
# Group by Cluster and calculate summary statistics
cluster_summary <- country1 %>%
  group_by(Cluster) %>%
  summarise(across(everything(), list(mean = mean, sd = sd), .names = "{.col}_{.fn}"))
```

Visualize the Clusters

```
ggplot(country1, aes(x = factor(Cluster), y = AveragScore, fill = factor(Cluster))) +
  geom_boxplot() +
  labs(title = "Average score Distribution Across Clusters", x = "Cluster", y = "AveragScore") +
  theme_minimal()
```



This plot visually represents the distribution of average scores for each of the three identified clusters.

1. Cluster 1 (Red Box): This cluster exhibits the highest average scores, with a median value well above 70. The interquartile range (IQR) is fairly narrow, suggesting that the countries in this cluster have similar high scores. This indicates that these countries are performing well in terms of overall prosperity indicators.
2. Cluster 2 (Green Box): Countries in Cluster 2 have an average score around 60, placing them in the middle range compared to the other clusters. The distribution shows greater variability compared to Cluster 1, as evidenced by a wider IQR. This suggests a mix of performance levels, with some countries scoring lower than the average while others perform moderately better.
3. Cluster 3 (Blue Box): Cluster 3 shows the lowest average scores, with a median around 50. The box is notably lower on the scale, indicating that the countries in this cluster struggle more with prosperity indicators. There may also be a few outliers present, as indicated by the shorter whisker below the box.

Comparative analysis: The clear differences among the clusters suggest distinct profiles of prosperity among the countries. Cluster 1 consists of thriving nations (developed countries), Cluster 2 includes moderately prosperous countries (developing countries), and Cluster 3 contains those with more significant challenges in development (underdeveloped countries).

Implications for Policy-making: This clustering can inform policymakers by highlighting which countries may require more targeted interventions to enhance their prosperity levels. Understanding these distinctions allows for more tailored approaches to development.

```
data = read.csv("../data/global prosperity and development indicators.csv")

countries <- ne_countries(scale = "medium", returnclass = "sf")

data$Country <- str_trim(data$Country)

data$Cluster <- clusters
```

```
data <- data %>%
  mutate(Country = case_when(
    Country == "United States" ~ "United States of America",
    Country == "Korea" ~ "South Korea",
    Country == "North Korea" ~ "Dem. Rep. Korea",
    Country == "Ivory Coast" ~ "Côte d'Ivoire",
    Country == "Czech Republic" ~ "Czechia",
    Country == "Dominican Republic" ~ "Dominican Rep.",
    Country == "Bosnia and Herzegovina" ~ "Bosnia and Herz.",
    Country == "Cape Verde" ~ "Cabo Verde",
    Country == "São Tomé and Príncipe" ~ "São Tomé and Príncipe",
    Country == "Equatorial Guinea" ~ "Eq. Guinea",
    Country == "Swaziland" ~ "eSwatini",
    Country == "Democratic Republic of Congo" ~ "Dem. Rep. Congo",
    Country == "Central African Republic" ~ "Central African Rep.",
    Country == "South Sudan" ~ "S. Sudan",
    Country == "Lao PDR" ~ "Laos",
    Country == "Macedonia" ~ "North Macedonia",
    Country == "Syrian Arab Republic" ~ "Syria",
    TRUE ~ Country
  ))
```

```

data$Cluster <- factor(data$Cluster, levels = c("3", "2", "1"), labels = c("Underdeveloped", "Developing", "Developed"))

world_data <- left_join(countries, data, by = c("name" = "Country"))

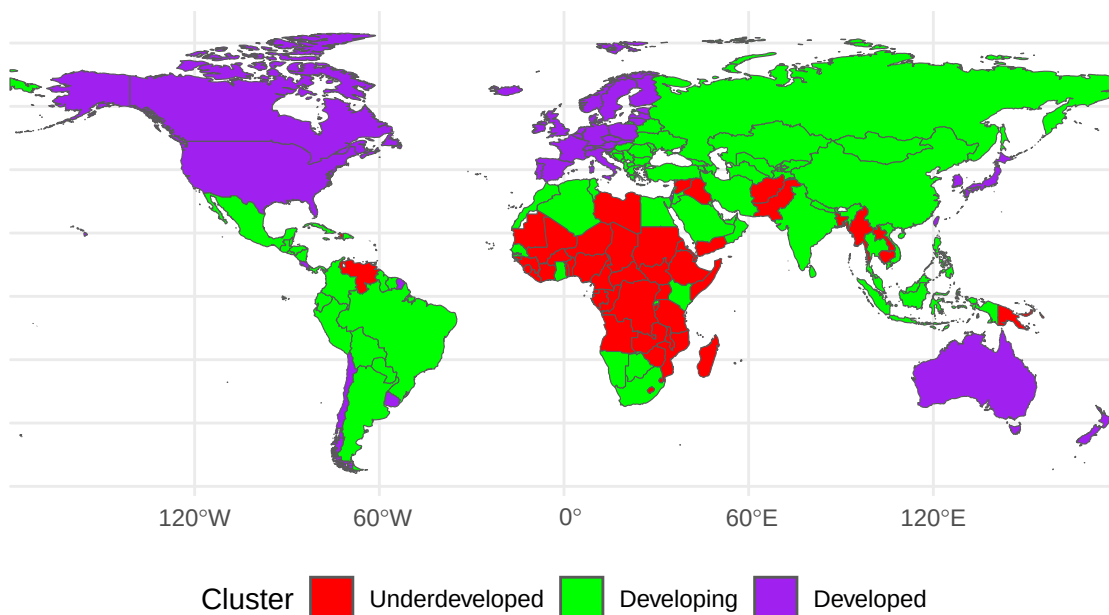
# Exclude rows with missing Cluster values
world_data <- world_data %>%
  filter(!is.na(Cluster))

cluster_colors <- c("Underdeveloped" = "red", "Developing" = "green", "Developed" = "purple")

ggplot(data = world_data) +
  geom_sf(aes(fill = Cluster)) +
  scale_fill_manual(values = cluster_colors, name = "Cluster", na.translate = FALSE) +
  theme_minimal() +
  labs(title = "Hierarchical Clustering Results on World Map",
       caption = "Source: Hierarchical Clustering of Global Prosperity Data") +
  theme(legend.position = "bottom")

```

Hierarchical Clustering Results on World Map



Source: Hierarchical Clustering of Global Prosperity Data

Conclusion

Based on the cluster validation, where APN, AD, ADM & FOM, where we can find values relatively close to 0, we can conclude that if we apply a hierarchical clustering, the average linkage method is the most suitable for our data, and if we observe that as per Cophenetic Correlation Coefficients shown above, we can also

conclude that for Hierarchical Clustering, the average linkage is the most suitable for our data (Cophenetic correlation: 0.7129493).

Research Question:

What methods exist for identifying and managing outliers in hierarchical clustering, and how might these be applied to countries with extreme prosperity scores?

1. Visual methods, including dendrogram analysis, where after performing hierarchical clustering, isolated clusters or small form clusters might be visible.
- 2.- Using Silhouette score to assess how well each country fits its assigned cluster, where a low silhouette score (close to -1) indicates that a country is poorly clustered and might be considered as an outlier. (this detection method was used above identifying 7 states with a negative score indicating a poor clustering and could be considered as outliers).
3.
 - Applying k-medoids after hierarchical clustering can help mitigate the influence of outliers; as k-medoids uses actual data points as the center of clusters (medoids), which makes it less sensitive to extreme values.